

xv6虚拟文件系统和硬件抽象层的设计与实现

赛题：2026年全国大学生计算机系统能力大赛-操作系统设计赛(全国)-OS功能挑战赛道

团队：tjuer__

成员：__陈权__ (VFS/ext2/FAT32)、____ (HAL/RISC-V/LoongArch)

日期：2026年6月26日

第一章 项目概述

1.1 项目背景

xv6-riscv是MIT开发的教学操作系统，运行于RISC-V平台，被广泛用于高校操作系统课程。其标准第5版存在两个核心局限：

- 单一文件系统**：仅支持自带的xv6文件系统（类Unix V6），无法与ext2、FAT32等主流文件系统交互
- 单一CPU架构**：内核代码与RISC-V紧耦合，移植到其他架构需大量重写

这使得xv6在教学中难以展示"Linux是如何同时支持ext4/xfs/btrfs的"以及"操作系统如何跨平台运行"等关键概念。

本项目为xv6-riscv第5版扩充**虚拟文件系统（VFS）**和**硬件抽象层（HAL）**，使其以最精简的代码同时支持三种文件系统和两种CPU架构，打造一个设计精巧的小型教学操作系统。

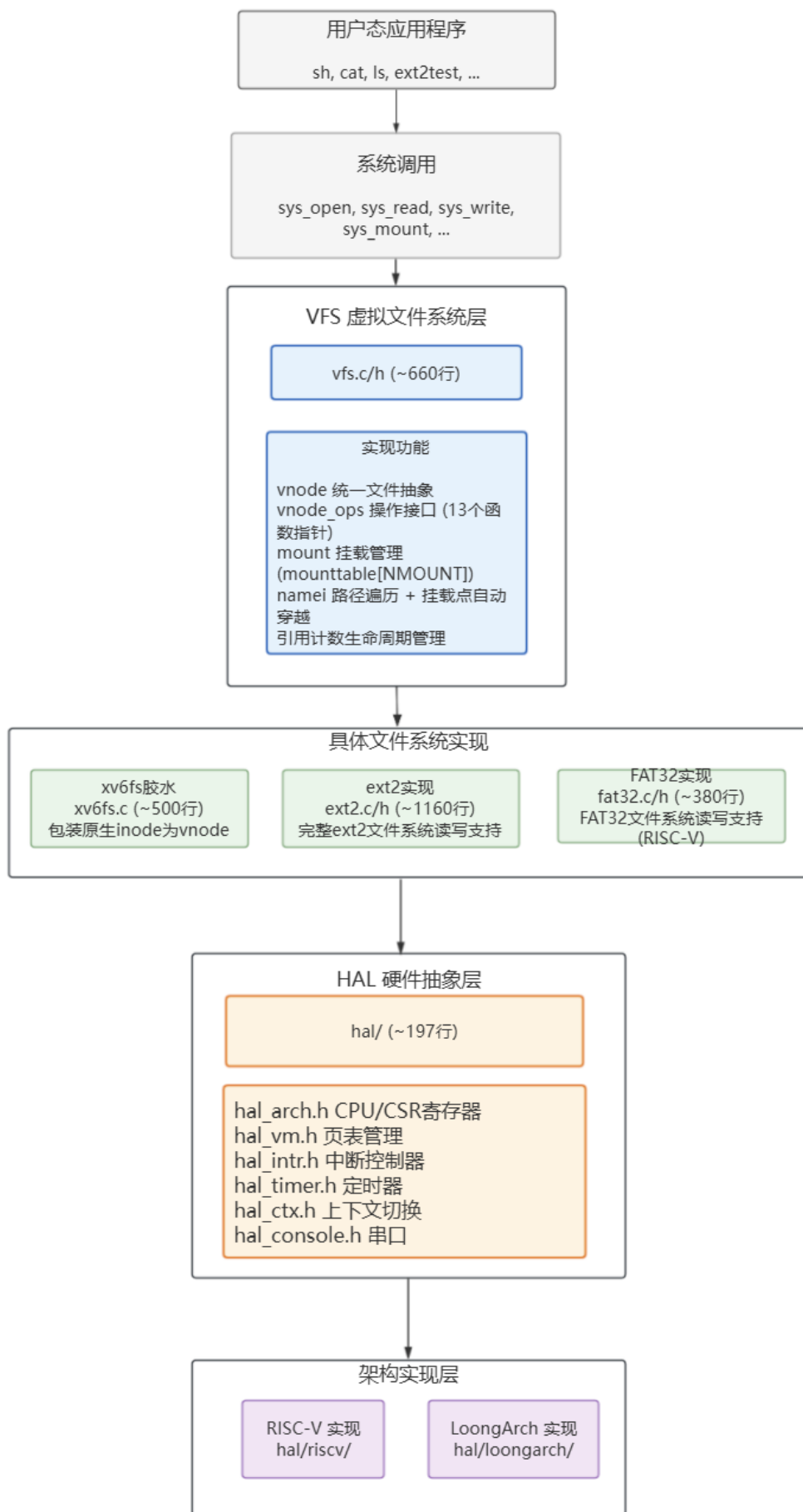
1.2 项目目标

编号	目标	说明
G1	同时支持3种文件系统	xv6fs（原生）、ext2（Linux早期标准）、FAT32（U盘通用）
G2	同时支持2种CPU架构	RISC-V（已验证）、LoongArch（国产自主架构）
G3	通过所有测例	xv6原生usertests + 赛方测试用例 + 自定义综合测试
G4	代码尽量少	对比原xv6-rev5，核心新增控制在~2000行以内
G5	运行稳定	无panic、无内存泄漏、并发安全

1.3 文档结构导览

章节	内容	读者对象
第一章	项目概述（本文）	所有读者
第二章	VFS设计详解	想理解虚拟文件系统核心算法的读者
第三章	ext2适配详解	想理解具体文件系统如何接入VFS的读者
第四章	xv6fs胶水层	想理解"最小侵入"策略的读者
第五章	HAL硬件抽象层	想理解跨平台抽象设计的读者
第六章	FAT32适配	想理解FAT32实现要点的读者
第七章	测试与验证	评审老师、想复现结果的读者
第八章	设计精巧度分析	评审老师（评分要点"设计精巧度"的直接支撑）
第九章	挑战与展望	所有读者

1.4 总体架构



架构核心思想： VFS层通过统一vnode抽象隔离上层系统调用与具体文件系统；HAL层通过统一接口头文件隔离内核核心逻辑与CPU架构。两层抽象互为正交——添加新文件系统不需要懂CPU，移植新架构不需要懂文件系统。

1.5 团队分工

成员	负责模块	具体内容
陈权	VFS + 多文件系统	VFS核心设计、xv6fs胶水层、ext2完整实现、FAT32实现、综合测试
杨子游	HAL + 多架构	HAL接口设计、RISC-V平台实现、LoongArch平台移植

1.6 功能模块一览

功能模块	说明
VFS核心层	mount/umount、namei路径遍历、挂载点穿越、"."跨FS返回
xv6fs胶水层	原生inode零修改包装为vnode
ext2文件系统	完整vnode_ops（13个函数），支持文件/目录/硬链接/设备节点
FAT32文件系统	基本读写框架（仅RISC-V），8.3短名，无长文件名
RISC-V HAL	启动、页表、中断/定时器、串口、virtio磁盘驱动
LoongArch HAL	启动、TLB软重填、中断/定时器、串口、ramdisk

1.7 代码量统计

类别	文件	行数
VFS核心	kernel/vfs.c + kernel/vfs.h	553 + 111 = 664行
ext2实现	kernel/ext2.c + kernel/ext2.h	1037 + 128 = 1165行
xv6fs胶水	kernel/xv6fs.c	499行
FAT32	kernel/fat32.c + kernel/fat32.h	301 + 76 = 377行
HAL接口层	hal/hal.h + hal/hal_*.h (7个接口头文件)	~180行
RISC-V HAL	hal/riscv/ (11个文件)	~1460行
LoongArch HAL	hal/loongarch/ (13个文件)	~1390行
上层适配	kernel/sysfile.c (386行) + kernel/file.c (163行)	549行
用户测试	user/ext2test*.c + user/fat32test.c	~380行
合计新增		~4340行
原xv6-rev5基线		~10000行
增幅		~43%

第二章 VFS虚拟文件系统设计

2.1 设计动机与目标

2.1.1 原xv6的局限

xv6-riscv-rev5 的文件系统代码（kernel/fs.c、kernel/log.c）存在三个固有限制：

- 1. 硬编码文件系统类型：所有文件操作直接操作 struct inode，inode 管理（ialloc、iget、iput）、目录逻辑（dirlookup、dirlink）、块映射（bmap）全部假设底层是 xv6 原生 FS 格式。要支持 ext2，必须重写整个文件操作栈。

- 2. **无挂载点概念**：只有一个根文件系统，无法像 Linux 那样在 `/mnt` 上挂载第二块磁盘。根 inode 从超级块直接读取，不存在 "不同设备上不同文件系统" 的抽象。
- 3. **路径遍历与 FS 实现耦合**：`namei` / `nameiparent` 内联调用 `dirlookup` (xv6fs 特有)，无法遍历 ext2 目录项。

2.1.2 VFS 的设计目标

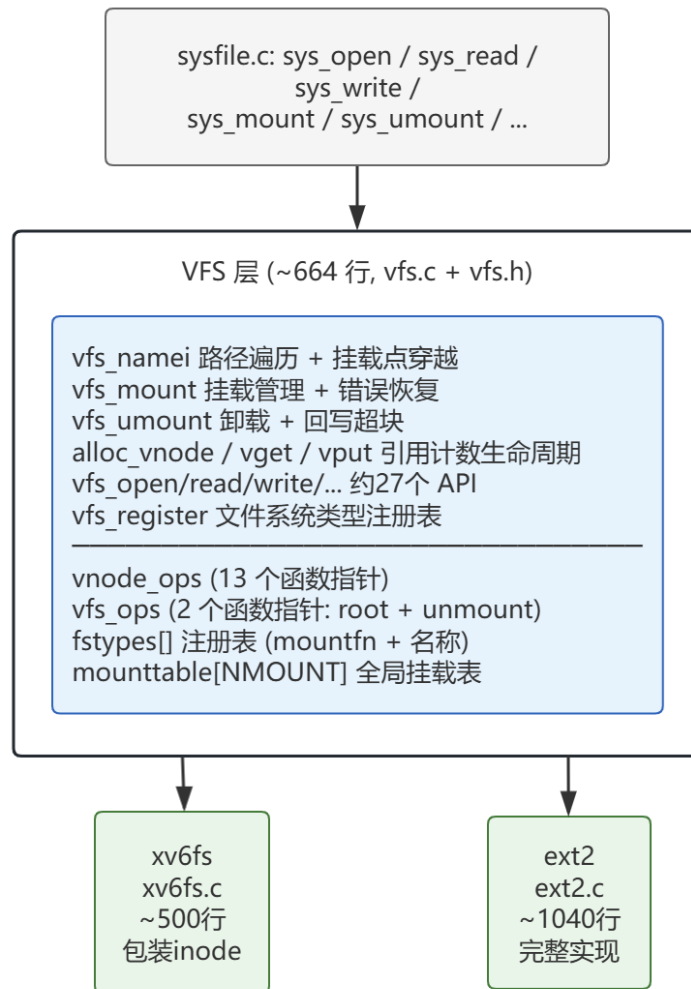
目标	具体含义
统一接口	所有文件操作通过 <code>vnode_ops</code> 虚拟表调用，系统调用不感知底层 FS 类型
多 FS 支持	可同时挂载 xv6fs (<code>root</code>) 和 ext2 (<code>/mnt</code>)，路径遍历自动在挂载点间穿越
最小侵入	<code>kernel/fs.c</code> 和 <code>kernel/log.c</code> 一行不改，xv6fs 作为独立胶水层包装原 inode
代码精简	<code>vfs.c</code> + <code>vfs.h</code> 合计约 664 行，符合教学操作系统定位
依赖倒转	VFS 层不依赖具体 FS 实现——FS 通过 <code>vfs_register</code> 注册自身，VFS 只认 <code>vnode_ops</code> 接口

2.1.3 设计原则

- **不随复杂度膨胀**：宁可一个 `vnode` 统一抽象，不要 `dentry` / `inode` / `address_space` 多层解体
- **引用计数驱动**：所有 `vnode` 生命周期由 `ref` 精确管理，`vput` 触发延迟销毁
- **错误路径可恢复**：`mount` 失败时完整回滚，不泄漏内存或锁

2.2 整体架构

VFS 层嵌于系统调用与具体文件系统之间，对上提供约 27 个 `vfs_*` API (全部在 `kernel/vfs.h` 中声明)，对下通过两套虚拟表 `vnode_ops` (文件级) 和 `vfs_ops` (挂载级) 驱动具体实现。横向通过全局 `mounttable[NMOUNT]` 管理最多 8 个挂载点，`namei` 路径遍历在挂载点间自动穿越。



关键交互路径：

- **上对 sysfile.c:** 所有系统调用通过 `vfs_open` / `vfs_read` / `vfs_write` / `vfs_close` / `vfs_mount` / `vfs_umount` 等薄包装进入 VFS
- **下对具体 FS:** VFS 不持有 FS 实现的任何头文件引用——仅通过 `fstypes[]` 注册表在 mount 时调用 `mountfn(dev)`，之后通过 `vnode_ops` 回调
- **横向挂载管理:** `namei` 中每步 `lookup` 后调用 `cross_mount` 检查是否为挂载点，自动穿越到下级文件系统

2.3 核心数据结构

2.3.1 vnode — 统一文件抽象

```

1 struct vnode {
2     uint type;           // V_FILE(1), V_DIR(2), V_DEV(3)
3     uint dev;           // 所属设备的设备号, 从 mp->dev 继承
4     int major;          // V_DEV 的主设备号
5     int minor;          // V_DEV 的次设备号
6     struct vnode_ops *ops; // 操作虚拟表, 由所属 FS 在创建时填充
7     struct mount *mp;    // 所属挂载点; ref==0 时置 NULL
8     int ref;            // 引用计数: vget 递增, vput 递减到 0 触发销毁
9     struct sleeplock lock; // 保护 vnode 读写操作的睡眠锁
10    uint inum;           // FS 内部 inode 号, 仅在所属 mp 内有意义
11    uint size;           // 缓存的文件大小, 由 FS 后端维护 (O_APPEND 等)
12    void *priv;          // FS 私有数据指针 (如 xv6fs 的 struct inode*)
13 };
  
```

字段详解：

字段	作用	生命周期
type	区分文件/目录/设备节点	alloc_vnode 置 0, FS 的 create/lookup 设置为 V_FILE/V_DIR/V_DEV
dev	用于 mountpoint 匹配和 I/O 路由	从 mp->dev 继承, vnode 销毁时不变
inum	配合 dev 在系统内唯一标识一个文件	FS 的 create/lookup 填入
ref	防止正在使用的 vnode 被回收	alloc_vnode 置 1, vget / vput 原子操作
ops	解耦 VFS 与具体 FS: VFS 只调 ops, 不关心实现	FS 的 create/lookup 填充, vput 销毁前置 NULL
mp	反向指针, 用于 vfs_find_mountpoint 和 vfs_link 跨FS检查	由 vfs_mount 或 FS create 设置
priv	避免 vnode 膨胀——FS 自行分配私有结构挂载于此	FS create/lookup 分配, ops->destroy 释放
size	缓存文件大小, file_write 的 O_APPEND 依赖此字段	FS 的 read/write/truncate 维护
lock	串行化对同一 vnode 的并发读写 (如 file_read/file_write)	初始化时 initsleeplock, 销毁时隐式释放

2.3.2 vnode_ops — 文件操作接口 (13 个函数指针)

vnode_ops 是 VFS 设计的核心抽象, 每个具体文件系统实现这 13 个函数, VFS 通过 vop_* 宏间接调用。

序号	函数指针	签名	功能
1	lookup	int(vnode*, char*, vnode**)	在目录中查找 name, 返回目标 vnode (ref 已持有)。失败返回非0
2	mknod	int(vnode*, char*, int, int)	在目录中创建设备节点 (major, minor)
3	read	int(vnode*, uint64, int, uint)	从 off 偏移读 n 字节到用户地址 buf; 返回实际读取字节数
4	write	int(vnode*, uint64, int, uint)	将用户地址 buf 的 n 字节写入 off 偏移; 返回实际写入字节数
5	stat	int(vnode*, uint64)	填充 struct stat 到用户地址 addr
6	readdir	int(vnode*, uint64, uint)	从 off 偏移读一个目录项到用户地址 buf; 返回下一偏移
7	create	int(vnode*, char*, short, vnode**)	在 dir 中创建 type 类型的 inode; 失败返回非0
8	unlink	int(vnode*, char*)	从 dir 移除 name; 若 nlink==0 则释放 inode (可能延迟到 vput)
9	mkdir	int(vnode*, char*)	在 dir 中创建子目录 (含 "." 和 "..")
10	link	int(vnode*, char*, vnode*)	将 old vnode 硬链接到 dir 下名为 name
11	read_kernel	int(vnode*, uint64, int, uint)	内核态读——直接写入内核地址 buf, 避免 copyout 开销 (exec 用)
12	truncate	int(vnode*)	释放所有数据块, 设置 size 为 0; 仅 V_FILE
13	destroy	void(void*)	释放 vnode->priv 指向的 FS 私有数据, 在 vput 的锁外延迟调用

VFS 层通过 VOP_* 宏简化调用:

```
1 #define VOP_LOOKUP(v,n,r)      (v)->ops->lookup(v,n,r)
2 #define VOP_READ(v,a,n,o)      (v)->ops->read(v,a,n,o)
3 #define VOP_WRITE(v,a,n,o)     (v)->ops->write(v,a,n,o)
4 // ... 共 13 个宏
```

接口设计原则:

- read / write 使用 uint64 作为用户地址 (兼容 64 位指针, 内核态通过 copyin / copyout 访问)
- read_kernel 以直接内存访问替代 readi, 消除 exec 加载 ELF 时的 double-copy 开销
- 所有创建操作 (create / mkdir / mknod) 在持有 vn_lock 下调用, 保证目录完整性

2.3.3 vfs_ops — 挂载级操作接口

```
1 struct vfs_ops {
2     struct vnode* (*root)(struct mount*); // 返回本 FS 的根 vnode (ref 已持有)
3     void (*unmount)(struct mount*);       // 回写超块 + 释放 FS 私有挂载数据
4 };
```

vfs_ops 只定义了两个函数指针, 比 vnode_ops 简单得多。root 在 mount 完成时调用一次, unmount 在 umount 时调用。

2.3.4 mount 与 mounttable

```
1 struct mount {
2     int valid;                // 1 表示已成功注册到 mounttable (原子状态标记)
3     struct vfs_ops *ops;      // root() + unmount()
4     struct vnode *root;       // 本 FS 的根 vnode (ref 永久持有)
5     struct vnode *mountpoint; // 挂载点 vnode (父 FS 的目录 vnode); 根挂载为 NULL
6     uint dev;                 // 本挂载涉及的所有 I/O 的目标设备号
7     char path[128];           // 挂载路径字符串 (调试用; 实际遍历用 mountpoint 指针)
8     void *priv;               // FS 私有挂载数据 (如 ext2 的超块缓冲区)
9 };
```

挂载表是全局数组:

```
1 struct mount *mounttable[NMOUNT]; // NMOUNT=8
2 int nmount;                        // 当前活跃挂载数
```

示例布局 (同时挂载 xv6fs 和 ext2) :

```
1 mounttable[0] = { valid=1, dev=0, root=vA, mountpoint=NULL, path="/" } // xv6fs 根挂载
2 mounttable[1] = { valid=1, dev=1, root=vB, mountpoint=vC, path="/mnt" } // ext2 挂载在 /mnt
```

其中 vA 是 xv6fs 根 inode 的 vnode, vB 是 ext2 根 inode 的 vnode, vC 是 xv6fs 中 /mnt 目录的 vnode。

2.4 路径遍历算法

2.4.1 vfs_namei 完整流程

`vfs_namei` 是 VFS 的——它将路径字符串解析为目标 vnode, 并在每次遍历目录项后自动穿越挂载点。函数约 45 行, 逻辑如下:

```
1 vfs_namei(path):
2     1. path_start(path, &rest) // '/' → vget(mounttable[0]->root)
3     // 否则 → vget(p->cwd)
4     2. while (rest = skipelem(rest, name)) != 0:
5         vn_lock(vp)
6         if vp->type != V_DIR → 失败, 返回 NULL
7
8         // "." 跨 FS 特殊处理 (见 2.4.3)
9         if name=="." && vp 是其 FS 的 root:
10             mpoint = vget(vp->mp->mountpoint)
11             vn_unlock(vp); vput(vp)
12             vn_lock(mpoint); mpoint->ops->lookup(mpoint, ".", &next)
13             vn_unlock(mpoint); vput(mpoint)
14             vp = next
15             continue
16
17         // 正常 lookup
18         vp->ops->lookup(vp, name, &next) // FS 后端查找目录项, 返回 vnode
19         vn_unlock(vp); vput(vp)
20
21         vp = cross_mount(next) // 检查下一跳是否为挂载点, 若是则穿越
22     3. return vp              // ref 已持有, 调用方负责 vput
```

关键细节:

- `path_start` 返回的 vnode 已持有 ref (通过 `vget`), 确保遍历期间不被回收
- 每步 `lookup` 返回的 vnode 的 ref 由 FS 后端在 `lookup` 内部持有 (对应 `alloc_vnode` 语义)
- 逐级 `vput(vp)` 释放前一级 vnode, 保证引用计数精确平衡
- `cross_mount` 消费传入的 ref, 返回新 ref, 无需额外引用计数操作

2.4.2 挂载点穿越 (cross_mount)

```
1 static struct vnode*
2 cross_mount(struct vnode *vp)
3 {
4     struct mount *submp;
5     submp = vfs_lookup_mount(vp);
6     if(submp){
7         vput(vp);
8         return vget(submp->root);    // 返回挂载在 vp 上的 FS 的根 vnode
9     }
10    return vp;                        // 非挂载点，原样返回
11 }
```

`vfs_lookup_mount` 遍历 `mounttable[1..nmount-1]` (跳过根挂载)，用 `dev+inum` 匹配：

```
1 if(mounttable[i]->mountpoint &&
2     mounttable[i]->mountpoint->dev == vp->dev &&
3     mounttable[i]->mountpoint->inum == vp->inum)
4     return mounttable[i];
```

为什么用 dev+inum 而非指针比较？

不同 FS 对同一个 inode 调用 `vget` (或 `xv6fs` 的 `iget`) 可能返回不同地址的 **vnode 对象** (`vnode_table` 中的不同槽位)，但 `dev` 和 `inum` 在系统内是全局唯一的 (`dev` 标识设备，`inum` 是 FS 内 inode 号)。用 `dev+inum` 匹配保证了正确性，避免了因缓存复用导致的指针失效问题。

2.4.3 "." 跨文件系统处理

问题场景： `ext2` 挂载在 `xv6fs` 的 `/mnt` 上。当用户在 `ext2` 内执行 `cd ..` 时，应该从 `ext2` 回到 `xv6fs` 的 `/mnt` 目录，而非 `ext2` 自己的根目录。

实现逻辑 (嵌入在 `namei` 的循环中)：

```
1 if(strncmp(name, "..", 15) == 0 &&
2     vp->mp && vp->mp->mountpoint &&
3     vp->dev == vp->mp->root->dev &&
4     vp->inum == vp->mp->root->inum)
5 {
6     struct vnode *mpoint = vget(vp->mp->mountpoint);
7     vn_unlock(vp);
8     vput(vp);
9     vn_lock(mpoint);
10    mpoint->ops->lookup(mpoint, "..", &next);
11    vn_unlock(mpoint);
12    vput(mpoint);
13    vp = next;
14    continue;
15 }
```

图解：

```

1  xv6fs:
2      / → /mnt (vnode C) — "." → / (vnode A)
3      |
4  ext2:      | 挂载于此
5      /ext2root (vnode B) — "." → ?
6
7  处理流程:
8      cd .. 在 ext2 根目录执行
9      → 检测 vB.dev==mp->root->dev && vB.inum==mp->root->inum
10     → 是 root → 穿越到 mp->mountpoint (vC, xv6fs 的 /mnt)
11     → 在 xv6fs 中 lookup(mpoint, ".") → 返回 / (vA)
12     → 最终结果: vA (xv6fs 根目录) ✓

```

与 Linux 的对比: Linux 通过 `dentry` 的 `d_parent` 指针和 `follow_up` 实现相同语义。本 `vnode` 实现比 Linux 简化——不维护双向父子链接，仅在 `".."` 查询时动态穿越。

2.4.4 vfs_nameparent

`vfs_nameparent` 是 `vfs_namei` 的变体，用于 `O_CREATE` 路径：

```

1  vfs_nameparent("/mnt/newfile", name):
2      → 逐级遍历到 /mnt 的 vnode
3      → 保存最终分量 "newfile" 到 name
4      → 返回 /mnt 的 vnode (ref 持有)

```

调用方拿到父目录 `vnode` 后，在其上调用 `VOP_CREATE(dir, name, ...)` 创建新文件。这避免了先 `namei` 再解出父目录的双重遍历。

2.5 文件系统注册与挂载

2.5.1 vfs_register — 注册文件系统类型

VFS 维护一个静态注册表：

```

1  struct {
2      struct mount* (*mountfn)(uint);    // mount 入口函数: 接收 dev, 返回 mount 结构
3      char name[16];                     // FS 类型名称 ("xv6fs", "ext2")
4  } fstypes[8];
5  int nfstype;    // 已注册数量, 上限 8

```

注册接口极其简洁：

```

1  void vfs_register(struct mount* (*fn)(uint), char *name)
2  {
3      if(nfstype >= 8) panic("vfs_register: too many fstypes");
4      fstypes[nfstype].mountfn = fn;
5      safestrncpy(fstypes[nfstype].name, name, 16);
6      nfstype++;
7  }

```

`vfs_init` 在启动时调用两次注册：

```

1  vfs_register(xv6fs_mount, "xv6fs");
2  vfs_register(ext2_mount, "ext2");

```

添加新 FS (如 FAT32) 只需一行 `vfs_register(fat32_mount, "fat32")`，无需修改任何 VFS 内部逻辑。

2.5.2 vfs_mount — 挂载完整流程

`vfs_mount` 实现 7 个步骤，每步失败均有对应回滚：

```
1  vfs_mount(path, dev, fstype):
2      1. 重复挂载检查
3      for i in 0..nmount-1:
4          if mounttable[i]->path == path → return 0 （同名路径已挂载）
5
6      2. 查找 FS 类型
7      for i in 0..nfstype-1:
8          if fstypes[i].name == fstype:
9              mp = fstypes[i].mountfn(dev) // 调用 xv6fs_mount / ext2_mount
10         if mp == 0 → return 0
11
12     3. 解析挂载点 vnode
13     if path == "/":
14         mp->mountpoint = NULL
15     else:
16         mpoint = vfs_namei(path)
17         if mpoint==NULL || mpoint->type!=V_DIR:
18             → 回滚: vput(mpoint); mp->ops->unmount(mp); kfree(mp); return 0
19
20     4. 重复挂载检查（同一 vnode）
21     for i in 1..nmount-1: // 跳过 mounttable[0]
22         if mounttable[i]->mountpoint->dev==mpoint->dev &&
23             mounttable[i]->mountpoint->inum==mpoint->inum:
24             → 回滚: vput(mpoint); mp->ops->unmount(mp); kfree(mp); return 0
25
26     5. 填充 mount 结构
27     mp->mountpoint = mpoint
28     safestrcpy(mp->path, path, 128)
29     mp->dev = dev
30
31     6. 容量检查
32     if nmount >= NMOUNT:
33         → 回滚: vput(mpoint); mp->ops->unmount(mp); kfree(mp); return 0
34
35     7. 注册到 mounttable
36     mounttable[nmount] = mp
37     nmount++
38     mp->valid = 1
39     return mp
```

错误恢复路径的关键：

- 步骤 3~6 的失败回滚都执行 `mp->ops->unmount(mp) + kfree(mp)` ——因为 `mountfn` 已在步骤 2 中分配了 mount 结构和 FS 私有数据
- `vput(mpoint)` 释放步骤 3 获取的挂载点 vnode
- 不回滚 `fstypes[]` ——FS 类型注册是全局的，一次注册永久有效

2.5.3 vfs_umount — 卸载流程

```
1  int vfs_umount(char *path)
2  {
3      // 1. 在 mounttable 中按 path 字符串查找
4      for(idx=0; idx<nmount; idx++)
5          if(strncmp(mounttable[idx]->path, path, 128)==0)
6              mp = mounttable[idx];
7
8      // 2. 安全检查
9      if(mp==0 || !mp->valid) return -1;
10     if(idx == 0) return -1; // 根挂载不可卸载
11 }
```

```

12 // 3. 调用 FS 的 unmount 回调（回写超块、释放私有数据）
13 if(mp->ops && mp->ops->unmount)
14     mp->ops->unmount(mp);
15
16 mp->valid = 0;
17
18 // 4. mounttable 前移压缩（O(n) 删除，保留顺序）
19 for(i=idx; i<nmount-1; i++)
20     mounttable[i] = mounttable[i+1];
21 mounttable[nmount-1] = 0;
22 nmount--;
23
24 // 5. 释放挂载点 vnode
25 if(mp->mountpoint) vput(mp->mountpoint);
26
27 // 6. 释放 mount 结构体
28 kfree((void*)mp);
29 return 0;
30 }

```

注意事项：

- 查找用 `path` 字符串而非 `dev` ——两个设备可能有相同路径名称冲突
- 卸载不以 root vnode 的 ref 计数为条件——xv6 中假定调用 `umount` 时没有打开此 FS 上的文件（POSIX `umount` 的 `EBUSY` 检查省略）
- `mounttable` 删除通过数组前移实现， $O(n)$ 但 `NMOUNT=8` 时微不足道

2.6 vnode 生命周期管理

2.6.1 vnode 存储

vnode 使用静态数组 + 线性扫描分配，而非 Linux 的 slab 缓存：

```

1 struct vnode vnode_table[NVNODE]; // NVNODE=200, 约 200×80B ≈ 16KB
2 struct spinlock vnode_lock;      // 保护 ref 字段和分配状态

```

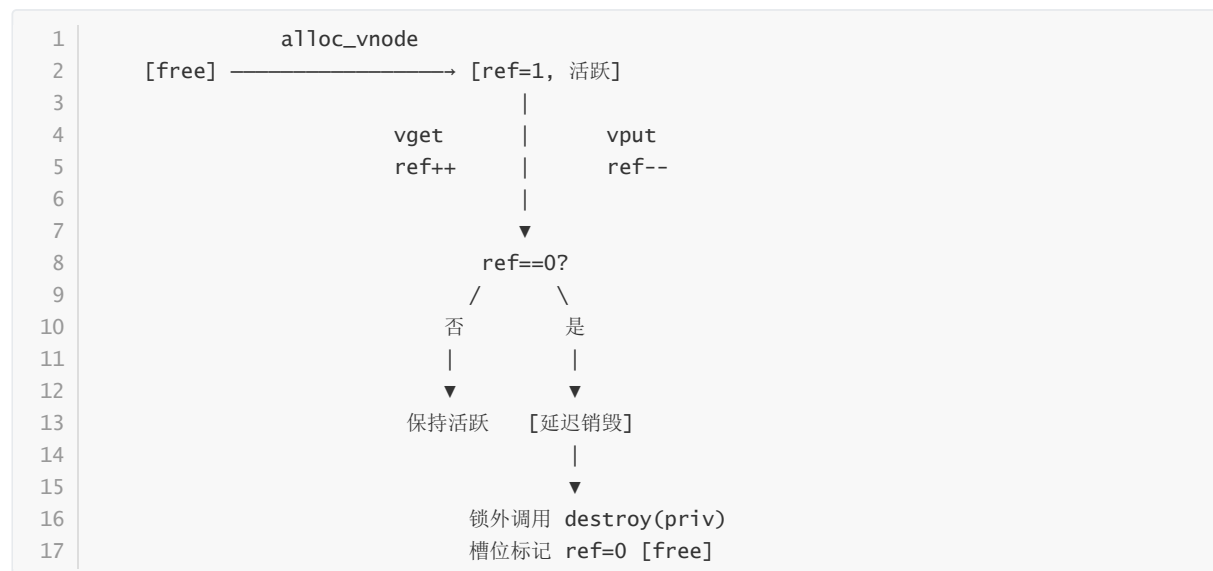
`alloc_vnode` 遍历 `vnode_table` 查找 `ref==0` 的空闲槽位，置 `ref=1` 并清零其他字段：

```

1 struct vnode* alloc_vnode(void)
2 {
3     acquire(&vnode_lock);
4     for(i = 0; i < NVNODE; i++){
5         if(vnode_table[i].ref == 0){
6             vnode_table[i].ref = 1;
7             vnode_table[i].type = 0;
8             vnode_table[i].ops = 0;
9             vnode_table[i].mp = 0;
10            vnode_table[i].priv = 0;
11            vnode_table[i].inum = 0;
12            vnode_table[i].major = 0;
13            vnode_table[i].minor = 0;
14            release(&vnode_lock);
15            return &vnode_table[i];
16        }
17    }
18    release(&vnode_lock);
19    return 0; // 返回 NULL 表示 vnode 池耗尽
20 }

```

2.6.2 引用计数状态机



- **vget**: `acquire(&vnnode_lock); vp->ref++; release(&vnnode_lock);` — 原子递增，调用者确保 vp 当前合法
- **vput**: `acquire(&vnnode_lock); vp->ref--; if(vp->ref==0) need_free=1; release(&vnnode_lock);` — ref 降到 0 时不立即销毁，而是在**释放自旋锁之后**异步调用 `destroy`

2.6.3 延迟销毁机制

```
1 void vput(struct vnode *vp)
2 {
3     void *priv;
4     void (*destroy)(void*);
5     int need_free = 0;
6
7     acquire(&vnnode_lock);
8     vp->ref--;
9     if(vp->ref == 0){
10         need_free = 1;
11         priv = vp->priv; // 快照私有数据
12         destroy = (vp->ops) ? vp->ops->destroy : 0; // 快照 destroy 函数
13         vp->priv = 0; // 清空槽位字段
14         vp->ops = 0;
15         vp->mp = 0;
16         vp->type = 0;
17     }
18     release(&vnnode_lock); // 先释放锁
19
20     if(need_free && destroy)
21         destroy(priv); // 再调用 destroy
22 }
```

为什么不能在线内调用 `destroy`?

`destroy(priv)` 最终会调用具体 FS 的清理函数（如 xv6fs 的 `iput` → `iunlockput`），后者可能涉及块分配/释放、日志写入、缓冲区操作——这些操作内部会获取其他锁。如果 `vput` 持有 `vnnode_lock` 自旋锁时调用 `destroy`，会导致：

1. **死锁**：`destroy` → `begin_op` → `log.lock`，同时另一线程持有 `log.lock` 等待 `vnnode_lock`
2. **自旋锁下睡眠**：块 I/O 可能触发 `sleep`，而在自旋锁下睡眠是违规的

延迟销毁通过在锁外调用 `destroy` 完美解决了这两个问题。这也是本设计中引用计数管理的核心诀窍。

2.7 跨文件系统操作限制

VFS 对跨挂载点的操作实施限制，保证 FS 内部数据结构的完整性。

2.7.1 硬链接限制 (vfs_link)

```
1 int vfs_link(struct vnode *old, struct vnode *newdir, char *name)
2 {
3     if(old->mp != newdir->mp) return -1; // 跨 FS 硬链接拒绝
4     // ...
5     return VOP_LINK(newdir, name, old);
6 }
```

硬链接的本质是同一个 FS 内部的 inode 别名——两个目录项指向同一个 inode 号。跨 FS 的硬链接无意义（ext2 的 inode 对 xv6fs 不可见）。Linux 对此返回 EXDEV 错误。

2.7.2 rename 的简化策略

xv6 VFS 不实现 `vfs_rename`——采用 `link` + `unlink` 的简化语义。这一方面是因为 xv6 原生的 `sys_rename` 不存在，另一方面 `rename` 的跨 FS 原子迁移非常复杂（涉及源 FS 的 orphan 和目的 FS 的数据拷贝），对教学操作系统而言代价过高。

2.7.3 其他跨 FS 操作

操作	跨 FS 行为	理由
<code>vfs_read</code> / <code>vfs_write</code>	允许（自动跟随 <code>vnode->ops</code> ）	只操作已打开的文件
<code>vfs_open</code>	允许（ <code>namei</code> 自动穿越挂载点）	路径遍历透明
<code>vfs_create</code>	不允许（必须在同一 <code>dir</code> 内）	<code>create</code> 在单个 <code>dir</code> <code>vnode</code> 上操作
<code>vfs_unlink</code>	不允许（删除在 <code>dir</code> 内完成）	同上

2.8 设计决策记录

决策	原因
<code>cwd</code> 用 <code>vnode*</code> 而非路径字符串	POSIX语义：目录 <code>rename</code> 后 <code>cwd</code> 仍指向同一 <code>vnode</code> ，路径字符串会失效
<code>priv</code> 用 <code>void*</code> 而非联合体	解耦 VFS 与具体 FS——添加新 FS 不动 <code>vnode</code> 结构
<code>NVNODE=200</code>	原 xv6 <code>NINODE=50</code> ； <code>vnode</code> 需更多：open file (<code>NFILE=128</code>) + <code>cwd</code> + <code>namei</code> 中间节点 + 跨 FS 挂载点
<code>mountpoint</code> 用 <code>dev+inum</code> 匹配	不同 <code>iget</code> 可能返回不同地址的 <code>vnode</code> 对象； <code>dev+inum</code> 在系统内全局唯一
<code>vput</code> 延迟销毁（锁外 <code>destroy</code> ）	避免 <code>destroy</code> 中的块分配/日志操作导致死锁或自旋锁下睡眠
线性扫描 <code>vnode</code> 缓存（非哈希）	代码精简； <code>NVNODE=200</code> 时 $O(N)$ 扫描开销可接受（ $\sim 0.2\mu s$ ）
<code>namei</code> 不使用 <code>dentry</code> 缓存	避免缓存失效的复杂性；路径遍历频率在 xv6 中很低
<code>mounttable</code> 固定数组	<code>NMOUNT=8</code> 足够（根 + 7 个额外 FS）；动态链表增加复杂而无实际收益
<code>umount</code> 不检查 <code>busy</code>	xv6 中调用 <code>umount</code> 前应确保无打开文件（简化实现，未来可加 <code>EBUSY</code> 检查）

2.9 涉及文件与代码量

文件	行数	角色
kernel/vfs.h	111行	vnode / vnode_ops / mount 类型定义, VOP_* 宏, API 声明
kernel/vfs.c	553行	namei 路径遍历, mount/umount, vnode 生命周期, 约27个 API
kernel/file.h	25行	struct file 中 f->ip 改为 struct vnode*
kernel/file.c	163行	file_read/file_write 调用 vfs_read/vfs_write; fileclose 调用 vput
kernel/sysfile.c	386行	所有 sys_* 调用 VFS API (sys_open, sys_mount, sys_umount 等)
kernel/proc.h	—	cwd 字段改为 struct vnode*
kernel/exec.c	—	exec 调用 vfs_namei + vfs_read_kernel 加载 ELF
kernel/fcntl.h	6行	O_CREATE / O_APPEND / O_TRUNC 等标志位定义

核心文件: `vfs.h` + `vfs.c` (664行, 包含全部VFS架构和逻辑)
关联文件: 5个上层调用方 (`sysfile.c`, `file.c`, `file.h`, `exec.c`, `proc.h`, `fcntl.h`)

第三章 ext2文件系统适配

3.1 ext2简介

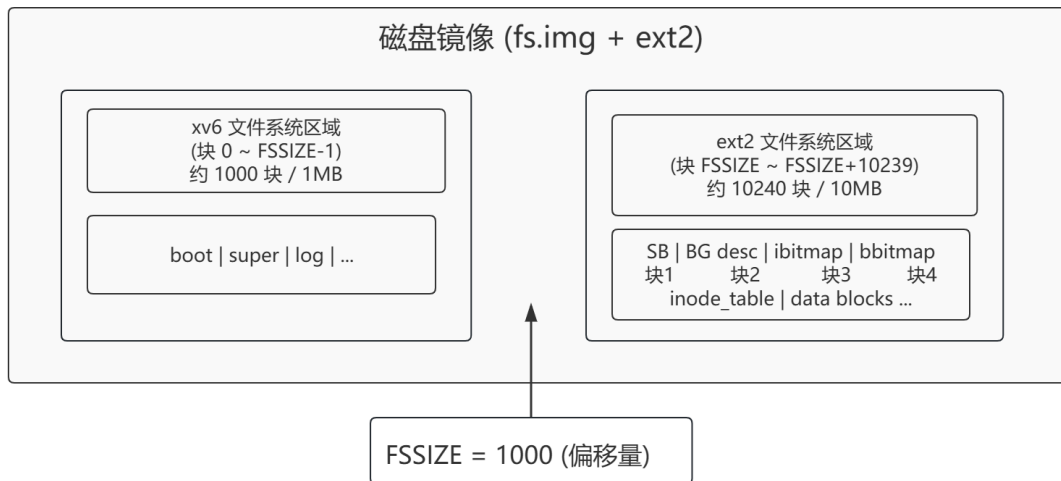
ext2 (Second Extended Filesystem) 是Linux早期标准的文件系统, 由Rémy Card于1993年设计。它采用经典的Unix文件系统布局——超级块、块组描述符、inode位图、块位图、inode表和 data blocks——结构清晰、文档丰富, 非常适合作为教学操作系统的"第二文件系统"范例。

本实现以约1000行C代码完整实现了ext2的基础读写能力(含文件/目录/设备节点的创建、读写、删除、硬链接和时间戳维护), 并通过vnode_ops接口无缝接入VFS层, 使得上层应用程序可以用与xv6fs完全相同的系统调用操作ext2文件系统。

3.2 适配策略

策略	说明
仅支持1024字节块	<code>s_log_block_size == 0</code> , 与xv6 BSIZE对齐, 直接复用buf-cache层 (bread/bwrite)
单磁盘分区布局	ext2分区紧接xv6 fs.img之后, 单磁盘双文件系统共享同一镜像
合并镜像偏移	ext2分区紧接xv6 fs.img之后, 块地址偏移 FSSIZE: <code>EBLK(mp, b) = b + mp->fs_offset</code>
128字节基础 inode	支持rev=0的128字节inode, 同时通过 <code>s_inode_size</code> 动态检测256字节inode
直接块 + 单间接块	<code>i_block[0..11]</code> + <code>i_block[12]</code> 单向间接, 最大文件268KB
不支持	扩展属性、ACL、日志、extent格式、双重/三重间接块、HTREE索引目录

合并镜像布局图:



3.3 数据流设计

用户态 read() 到ext2磁盘操作的完整链路:

```

1  用户程序: read(fd, buf, 100)
2      |
3      ▼
4  sys_read() [sysfile.c]
5      → fileread(f, addr, n) [file.c]
6          → vn_lock(vp)                                // 获取vnode睡眠锁, 串行化并发I/O
7          → VOP_READ(vp, addr, n, off)                  // 宏展开 → vp->ops->read(vp, addr, n, off)
8          → ext2_read(vp, buf, n, off) [ext2.c]
9              └─ 检查 off >= i_size → 返回0 (EOF)
10             └─ 调整 n 不超过文件末尾
11             └─ while(total < n):
12                 │   └─ ext2_bmap(priv, off/BSIZE, 0) // 逻辑块号→物理块号 (不分配, alloc=0)
13                 │   │   └─ lbn < 12 → 直接块, 读取 ino->i_bblock[lbn]
14                 │   │   └─ lbn 12..267 → 读间接块 → 查表 → 返回物理块号
15                 │   └─ blk == 0 → 稀疏空洞, 填充零到用户buf
16                 │   └─ bread(mp->dev, EBLK(mp, blk)) // 通过buf-cache读取磁盘块
17                 │   └─ copyout(p->pagetable, buf+total, bp->data+boff, len)
18                 └─ ext2_update_time(priv, 1, 0) // 更新atime
19      → vn_unlock(vp)
20      → 返回实际读取字节数

```

关键设计点:

- `ext2_bmap` 两次出境: `read`时 `alloc=0` (只读不分配), `write`时 `alloc=1` (按需分配)
- `bread/bwrite` 复用`xv6`的块缓存, 无需实现独立的磁盘I/O层
- `EBLK` 宏自动加上 `fs_offset` (`FSSIZE`), 实现合并镜像的透明偏移
- `vnode`锁在 `fileread/filewrite` 中获取, 保证`ext2`层内的操作是串行化的

3.4 核心数据结构

3.4.1 ext2_mount_priv — 挂载级私有数据

每次 `ext2_mount(dev)` 在堆上分配一个 `ext2_mount_priv` 结构体, 挂在 `mount->priv` 上:


```
1 struct ext2_mount_priv {
2     struct ext2_superblock sb;           // 内存中的超块副本（从块1读取）
3     struct ext2_bgdesc bg;              // 缓存唯一的块组描述符（从块2读取）
4     uint dev;                           // 设备号，传给 bread()/bwrite()
5     uint inode_size;                    // 磁盘inode大小（128或256字节）
6     uint fs_offset;                     // 合并镜像偏移量 = FSSIZE
7     struct sleeplock lock;              // 串行化 balloc/ialloc 位图操作
8     struct mount *mnt;                  // 反向指针，vnode → mp → mnt
9 };
```

各字段职责：

字段	来源	用途
sb	bread(dev, 1+FSSIZE)	全局用：s_free_blocks_count、s_free_inodes_count、s_blocks_count
bg	bread(dev, 2+FSSIZE)	定位：bg_inode_table、bg_block_bitmap、bg_inode_bitmap
inode_size	sb.s_inode_size（若为0则默认128）	ino_block / ino_offset 自适应计算
fs_offset	常量 FSSIZE	所有 bread / bwrite 的块号偏移
lock	初始化睡眠锁	保证 bitmap_alloc / bitmap_free 的原子性
mnt	ext2_mount 填充	vnode通过 priv->mp->mnt 回找挂载结构

3.4.2 ext2_vnode_priv — vnode级私有数据

每个ext2 vnode附着一个 ext2_vnode_priv，挂载在 vnode->priv 上：

```
1 struct ext2_vnode_priv {
2     struct ext2_inode_disk ino;          // 磁盘inode的内存缓存
3     struct ext2_mount_priv *mp;          // 所属挂载的私有数据（用于bread/balloc等）
4     uint inum;                           // 本inode的inode号
5     int dirty;                           // 1 = ino已修改，需要在destroy时回写
6     int orphaned;                        // 1 = unlink已将nlink降为0，destroy时真正释放
7 };
```

dirty标记的维护规则：

- ext2_update_time 更新 atime/mtime/ctime 时置1
- ext2_bmap 分配新块（间接块或数据块）时置1
- ext2_i_truncate 清空所有块指针和 i_blocks / i_size 后置1
- ext2_create 中 links_count 变化时置1
- ext2_write 里 i_size 扩展时置1
- ext2_destroy 中若非 orphaned 且 dirty，则 write_inode 回写磁盘

orphaned标记（见3.7.5节详解）：unlink将 links_count 降为0后置1，ext2_destroy 检测到此标记时执行块截断和inode释放。

3.5 块分配与回收（balloc/bfree/ialloc/ifree）

3.5.1 位图管理: bitmap_alloc / bitmap_free

块位图和inode位图均为一个1024字节块, 通过以下两个通用函数管理:

```
1 // 在位图块 bb 中分配bit [1..nbits-1] (bit 0保留)
2 static int bitmap_alloc(mp, bb, nbits)
3     1. bread(mp->dev, EBLK(mp, bb))
4     2. acquiresleep(&mp->lock) // 串行化并发分配
5     3. for i in 1 .. min(nbits, BSIZE*8):
6         if data[i/8] 的第(i%8)位 == 0:
7             data[i/8] |= (1 << (i%8)) // 置位
8             bwrite(bp)
9             releasesleep(&mp->lock)
10        return i
11    4. releasesleep(&mp->lock); brelse(bp); return -1 // 位图满
12
13 // 在位图块 bb 中释放bit n
14 static void bitmap_free(mp, bb, n)
15     1. bread → acquiresleep → 清零目标位 → bwrite → releasesleep
```

为什么要用睡眠锁? 位图更新可能涉及 `bwrite` (触发磁盘I/O), 而xv6的自旋锁不允许在持有期间睡眠。使用 `sleeplock` 既能保证互斥, 又允许 `bwrite` 内部的可能等待。

3.5.2 块分配流程 (ext2_balloc)

```
1 ext2_balloc(mp):
2     1. b = bitmap_alloc(mp, bg.bg_block_bitmap, sb.s_blocks_count)
3     2. acquiresleep(&mp->lock)
4     3. mp->sb.s_free_blocks_count-- // 超块空闲块计数 -1
5     4. ext2_sb_write(mp) // 回写超块 (bread → memmove → bwrite)
6     5. releasesleep(&mp->lock)
7     6. return b
```

3.5.3 块释放流程 (ext2_bfree)

```
1 ext2_bfree(mp, blockno):
2     1. bitmap_free(mp, bg.bg_block_bitmap, blockno)
3     2. acquiresleep → sb.s_free_blocks_count++ → ext2_sb_write → releasesleep
```

3.5.4 inode分配 (ext2_ialloc)

```
1 ext2_ialloc(mp, mode):
2     1. inum = bitmap_alloc(mp, bg.bg_inode_bitmap, sb.s_inodes_count)
3     2. bread 该 inode 所在的块
4     3. memset(ino, 0, sizeof(ext2_inode_disk)) // 清零所有字段
5     4. ino->i_mode = mode // 0x4000(目录) / 0x8000(文件) / 0x2000(设备)
6     5. ino->i_atime = ino->i_mtime = ino->i_ctime = ticks
7     6. bwrite + brelse
8     7. acquiresleep → sb.s_free_inodes_count-- → ext2_sb_write → releasesleep
9     8. return inum
```

失败回滚: 步骤2 bread失败 → 回滚 `bitmap_free(inode_bitmap, inum)`

3.5.5 inode释放 (ext2_ifree)

```
1 ext2_ifree(mp, inum):
2     1. bitmap_free(mp, bg.bg_inode_bitmap, inum)
3     2. acquiresleep → sb.s_free_inodes_count++ → ext2_sb_write → releasesleep
```

3.6 bmap：逻辑块→物理块映射

bmap是ext2所有I/O操作的必经之路，它将文件相对于块偏移量（lbn）转换为磁盘物理块号。

```
1 static uint ext2_bmap(struct ext2_vnode_priv *priv, uint lbn, int alloc)
```

参数	含义
priv	vnode私有数据，含inode缓存
lbn	文件内逻辑块号（0 = 第0块）
alloc	0 = 只读查找（read、stat、lookup用）；1 = 缺失时自动分配（write用）

寻址层次：

```
1 直接块区 (lbn 0..11):
2    ino->i_block[lbn] → 直接返回物理块号
3    若==0且alloc==1 → ext2_balloc → 写入i_block[lbn]
4                        i_blocks += 2 (一个1024字节块 = 两个512字节扇区)
5
6 单间接块区 (lbn 12..267):
7    1. 若 i_block[12]==0 且 alloc==1:
8      a. ext2_balloc 分配间接块本身
9      b. bread + memset(0, BSIZE) 清零整个间接块
10     c. i_blocks += 2
11    2. bread(indirect_block) 读取间接块
12    3. indirect[lbn-12] → 目标数据块的物理块号
13    4. 若==0且alloc==1 → ext2_balloc → 写入indirect[lbn-12]
14                        i_blocks += 2
15    5. 返回物理块号
16
17 双重/三重间接(lbn 268+):
18    return 0 (不支持)
```

范围	块数	容量
直接块(0-11)	12	12KB
单间接块(12-267)	256	256KB
合计	268	268KB

bmap失败回滚：

- 分配间接块后 bread 失败 → ext2_bfree(间接块) → i_block[12]=0 → i_blocks -= 2
- 分配数据块后调用方 bread 失败 → bmap本身返回值非0，调用方检测

3.7 目录操作详解

3.7.1 目录项格式

ext2目录是一个由变长 ext2_dir_entry 组成的线性列表，散布在目录的数据块中：

```
1 struct ext2_dir_entry {
2     uint    inode;        // inode号 (0 = 已删除的空位)
3     ushort  rec_len;      // 本条目总长度 (含padding)，必须被4整除
4     uchar   name_len;     // 真正名字长度 (不含NUL)
5     uchar   file_type;    // EXT2_FT_* 类型提示 (避免额外读inode)
6     char    name[];       // 变长名字 (长度 = name_len)，对齐到rec_len
7 };
```

关键规则：

- `rec_len` 包含名字后的空余字节 (padding) , 不允许条目跨块
- 第一个块的第一个条目以完整的 `rec_len = BSIZE` 开始
- 一个块末尾处若剩余空间不够最小条目 (8字节) , 前一条目的 `rec_len` 会延伸到块尾
- `file_type` 为快速类型判断提供提示: `EXT2_FT_REG=1`、`EXT2_FT_DIR=2`、`EXT2_FT_CHRDEV=3`

3.7.2 ext2_dir_lookup 查找

```
1 static int ext2_dir_lookup(struct vnode *dir, char *name, struct vnode **result)
```

逐块扫描目录内容, 对每个活跃条目 (`inode != 0`) 比较名字:

```
1 while(off < i_size):
2     blk = ext2_bmap(dp, off/BSIZE, 0)    // 不分配
3     bread(blk) → 逐条目:
4         if name匹配:
5             inum = de->inode
6             *result = ext2_iget(mp, inum)    // 读inode → 创建vnode (持有ref)
7             return 0
8         off += de->rec_len
9     return -1    // 未找到
```

性能特点: $O(N)$ 线性扫描——对于教学操作系统的目录规模 (通常 < 100个条目) 足够快。

3.7.3 ext2_dir_add 插入

```
1 static int ext2_dir_add(struct vnode *dir, char *name, uint inum, uchar ftype)
```

需求长度计算: `needlen = (sizeof(de) + strlen(name) + 3) & ~3` (4字节对齐)

三种插入策略 (按优先级) :

```
1 情况1: 复用已删除条目
2
3 | de(v0) | (空闲空间, reclen >= needlen) |
4 | inode=0 | |
5
6 处理: 直接填充 inode、name_len、file_type、name → bwrite
7 效果: 原条目的 rec_len 不变
8
9 情况2: 拆分条目padding
10
11 | de (活跃) | (padding, ≥ needlen) |
12 | real_len | reclen - real_len |
13
14 处理:
15 1. 原条目 rec_len 缩小为 real_len
16 2. 在 real_len 偏移处创建新条目
17 3. 新条目 rec_len = 原reclen - real_len
18 4. 填充新条目 → bwrite
19 效果: 原活跃条目不变, 其padding变为新活跃条目
20
21 情况3: 扩展目录 (新建一个块)
22 在目录末尾 expand one data block:
23 ext2_bmap(dp, off/BSIZE, 1) → new block
24 memset(bp->data, 0, BSIZE)
25 de = first entry; de->rec_len = BSIZE (单个条目占满整块)
26 填充 inode / name → bwrite
27 ino->i_size = off + BSIZE (扩展目录大小)
```



```

21 // → 若ref降为0, destroy()中检测到 orphaned, 执行真正释放
22 否则:
23     ino.i_ctime = ticks; write_inode // 仅更新时间戳

```

orphaned延迟释放的设计思想:

```

1 场景: 文件被进程A打开, 同时进程B unlink它
2
3 时间轴:
4  T1: open(fd, "/mnt/f") → vnode ref=2 (fd表 + namei临时)
5  T2: namei 退出 → vnode ref=1 (仅fd表)
6  T3: unlink("/mnt/f") → links_count降为0, orphaned=1
7                               → 此时不能立即释放块, 因为fd表还持有vnode引用
8  T4: close(fd) → vput → ref=0
9                               → destroy() 检测到 orphaned==1
10                               → ext2_i_truncate(priv) // 释放所有数据块
11                               → ext2_ifree(mp, inum) // 释放inode

```

关键优势:

- 安全的并发语义: open的文件在close前始终可读写, 即使已被unlink
- 避免悬垂指针: `links_count` 降为0但vnode存在期间, `inum` 仍在inode位图中标记为已用
- 与Linux语义一致: 符合POSIX "删除最后目录项时释放空间"的要求

3.8 时间戳管理

ext2维护三个POSIX时间戳 (以 `ticks` 为单位, xv6中1 tick ≈ 10ms) :

```

1 static void ext2_update_time(struct ext2_vnode_priv *priv, int access, int mod)

```

参数	更新字段	调用场合
<code>access=1, mod=0</code>	<code>i_atime</code>	read、lookup、read_kernel
<code>access=0, mod=1</code>	<code>i_mtime, i_ctime</code>	write、create、unlink、truncate
<code>access=1, mod=1</code>	<code>i_atime, i_mtime, i_ctime</code>	(未使用)

调用方分布在10个vnode_ops函数中:

- `ext2_read` / `ext2_read_kernel` → `ext2_update_time(priv, 1, 0)` (访问时间)
- `ext2_write` → `ext2_update_time(priv, 0, 1)` (修改时间)
- `ext2_lookup` → `ext2_update_time(dp, 1, 0)` (目录atime)
- `ext2_create` / `ext2_unlink` / `ext2_link` / `ext2_mknod` → `ext2_update_time(dp, 0, 1)` (目录mtime)

3.9 设备节点支持

设备节点的 `major/minor` 号编码存储在 `i_block[0]` 中:

```

1 // 写入 (ext2_mknod):
2 ino->i_block[0] = (major << 8) | (minor & 0xFF);
3
4 // 读出 (ext2_vnode_allo):
5 if(vp->type == V_DEV){
6     vp->major = (ino->i_block[0] >> 8) & 0xFF;
7     vp->minor = ino->i_block[0] & 0xFF;
8 }

```

设备节点类型为 `EXT2_INODE_CHRDEV` (字符设备, `i_mode` 高4位 = 0x2000), `i_links_count = 1`, 无数据块。创建时通过 `ext2_ialloc(mp, 0x2000)` 分配inode。

3.10 限制与兼容性

限制类别	具体限制	原因
寻址	最大文件 268KB（直接+单间接）	未实现双重/三重间接块
文件系统大小	≤10MB（1个块组）	仅实现单块组，s_blocks_per_group 上限8192
块大小	仅1024字节	与xv6 BSIZE对齐，不支持2048/4096字节块
inode大小	128或256字节	动态检测 s_inode_size，但仅操作前128字节
目录索引	线性扫描 O(N)	未实现HTREE索引目录
日志	无ext3/ext4日志	ext2本身无日志，崩溃恢复由上层负责
扩展属性	不支持	i_file_acl 忽略
ACL	不支持	i_dir_acl 忽略
符号链接	以普通文件处理	EXT2_INODE_SYMLNK 映射为 V_FILE
其他inode类型	FIFO、SOCK、BLKDEV 以普通文件处理	测试套件未覆盖

3.11 代码量统计

文件	行数	说明
kernel/ext2.h	128 行	磁盘结构定义（superblock、bgdesc、inode_disk、dir_entry）+ 常量
kernel/ext2.c	1037 行	全部逻辑实现（mount/umount + 13个vnode_ops + bmap + 分配器 + 目录操作 + 时间戳）
合计	1165 行	

各模块代码量分解：

模块	函数	行数
mount/umount	ext2_mount, ext2_unmount, ext2_root	~80
vnode管理	ext2_vnode_alloc, ext2_destroy, ext2_iget	~80
inode I/O	ino_block, ino_offset, read_inode, write_inode	~40
位图分配器	bitmap_alloc, bitmap_free	~30
块/inode分配回收	ext2_balloc, ext2_bfree, ext2_ialloc, ext2_ifree	~70
bmap	ext2_bmap	~60
目录操作	ext2_dir_lookup, ext2_dir_add, ext2_create, ext2_unlink, ext2_mkdir, ext2_link, ext2_mknod	~280
读写	ext2_read, ext2_write, ext2_read_kernel	~90
truncate	ext2_i_truncate, ext2_truncate	~35
stat/readdir/时间戳	ext2_stat, ext2_readdir, ext2_update_time	~70
vtable	ext2_vnops, ext2_vfsops	~30

第四章 xv6fs胶水层

4.1 设计原则：最小侵入

xv6fs胶水层的核心哲学是**零修改包装**——将xv6原生的 `struct inode` 及其操作函数（`fs.c` 中的 `iget`/`iput`/`readi`/`writei`/`dirlookup`/`dirlink`/`ialloc`/`itrunc`/`stati`）包装为VFS标准接口 `vnode_ops`，而不修改原生文件系统的任何一行代码。

原则	具体体现
fs.c 一行不改	原生文件系统代码（ <code>kernel/fs.c</code> 、 <code>kernel/log.c</code> 、 <code>kernel/fs.h</code> ）完全不动
薄适配层	xv6fs.c约499行，每个ops函数平均约38行
日志语义保持	<code>begin_op</code> / <code>end_op</code> 包裹所有修改操作，与原始sysfile.c行为一致
引用计数透传	xv6的 <code>iget</code> / <code>iput</code> 引用计数语义不变， <code>vnode ref</code> 通过 <code>xv6fs_vnode_priv</code> 间接持有inode
不加新锁	继续使用xv6的 <code>ilock</code> / <code>iunlock</code> ， <code>vnode</code> 锁由VFS层在进入ops之前获取

4.2 inode→vnode 映射

4.2.1 私有数据结构

```
1 struct xv6fs_vnode_priv {
2     struct inode *ip; // 指向xv6原生inode（已持有引用）
3 };
```

与ext2的 `ext2_vnode_priv` 相比极其简洁——因为xv6的原生inode已经包含所有元数据（`type`、`major`、`minor`、`inum`、`size`），无需缓存磁盘inode。

4.2.2 vnode工厂函数

```
1 static struct vnode* xv6fs_vnode_from_inode(struct inode *ip, uint dev, struct mount *mp)
```

字段映射表：

vnode字段	来源	说明
type	ip->type	T_DIR → V_DIR, T_DEVICE → V_DEV, 其他 → V_FILE
major	ip->major	设备主设备号
minor	ip->minor	设备次设备号
dev	参数 dev	从上级vnode或挂载结构继承
mp	参数 mp	所属挂载结构
inum	ip->inum	xv6 inode号 (1 = ROOTINO)
size	ip->size	文件大小缓存
priv	新分配的 xv6fs_vnode_priv	持有 ip 的引用
ops	&xv6fs_vnops	指向xv6fs的vnode_ops虚表

调用前必须持有 ilock(ip)：因为 ip->type、ip->major、ip->minor、ip->size 的读取需要锁保护。工厂函数在读取完这些字段后调用方释放锁。

4.2.3 引用计数协议

```
1      └ iget(dev,inum) → ref(ip)=1
2      | ilock(ip)
3      | xv6fs_vnode_from_inode(ip,dev,mp) → vnode ref=1, priv->ip=ip
4      | iunlock(ip)
5      |
6      | ... vnode 活跃期间 ...
7      | ip 的引用计数通过 priv->ip 间接持有
8      |
9      ▼
10     vput(vp) → ref(vp)==0 → xv6fs_destroy(priv):
11         └ begin_op()
12         └ iput(priv->ip) → ref(ip)--, 若0则释放inode+trunc块
13         └ end_op()
14         └ kfree(priv)
```

4.3 各ops实现要点

ops	实现策略	行数	关键细节
lookup	<code>ilock(dp) → dirlookup(dp, name, 0) → iunlock(dp) → xv6fs_vnode_from_inode</code>	~12	dirlookup返回的inode已持有引用，工厂函数消费
read	<code>ilock(ip) → readi(ip, 1, buf, off, n) → iunlock(ip)</code>	~8	<code>1</code> = 用户空间地址（readi内部调用copyout）
write	大写入拆分多个 <code>begin_op/ilock/writei/iunlock/end_op</code> 迭代	~25	单次writei受 <code>(MAXOPBLOCKS-1-1-2)/2*BSIZE</code> 日志配额限制，需循环
stat	<code>ilock → stati → iunlock → copyout</code>	~8	stati是xv6的串行化stat填充函数
readdir	<code>ilock → readi(ip, 0, &de, off, 14) → iunlock → 转换为vdirent → copyout</code>	~25	跳过 <code>inum==0</code> 的已删除条目；返回下一偏移
create	<code>begin_op/ilock/dirlookup检查重名/ialloc/dirlink/建"."/".."/iunlock/end_op</code>	~60	完整回滚逻辑：若"."或"..创建失败则清零父目录条目
unlink	<code>begin_op/ilock/dirlookup/isdirempty检查/ip->nlink--/iupdate/iunlockput/清零条目/iunlock/end_op</code>	~45	目录删除要求 <code>isdirempty</code> 通过； <code>iunlockput</code> 同时释放锁和引用
mkdir	委托 <code>xv6fs_create(dir, name, V_DIR, &new)</code>	~5	创建后立即 <code>vput(new)</code> （VFS层持有自己的ref）
link	<code>begin_op/ilock(old)/ip->nlink++/iupdate/iunlock/ilock(dp)/dirlink/iunlock/end_op</code>	~25	失败时回滚 <code>ip->nlink--</code>
mknod	<code>begin_op/ilock/dirlookup检查重名/ialloc(T_DEVICE)/设major+minor/iupdate/dirlink/iunlock/end_op</code>	~30	-
read_kernel	<code>ilock → readi(ip, 0, buf, off, n) → iunlock</code>	~8	<code>0</code> = 内核空间（直接memmove），exec加载ELF用
truncate	<code>begin_op/ilock/itrunc/iunlock/end_op</code>	~8	itrunc释放所有数据块， <code>vp->size = 0</code>
destroy	<code>begin_op/iput(ip)/end_op/kfree(priv)</code>	~10	iput含日志包裹，可能触发的itrunc/ifree也在日志内

4.4 代码量

文件	行数	说明
<code>kernel/xv6fs.c</code>	499行	全部胶水逻辑实现

模块	行数
数据结构与vnode工厂	~40
mount/root	~35
destroy（释放回调）	~12
vnode_ops（13个函数）	~210
vtable（虚表填充）	~20

注：xv6fs.c 不需要独立的 `.h` 文件——其接口就是 VFS 标准接口 `vnode_ops + vfs_ops`，所有类型定义已在 `vfs.h` 中声明。

第五章 HAL硬件抽象层

5.1 HAL设计目标

HAL（硬件抽象层）的核心使命是**让内核通用代码不感知CPU架构差异**，使得同一份 `kernel/` 源码能在RISC-V和LoongArch上编译运行，无需任何修改。

5.1.1 设计原则

原则	说明
三明治架构	顶层 <code>kernel/</code> 只调用 <code>hal_*</code> 前缀接口 → 中层 <code>hal/hal_*.h</code> 声明统一API → 底层 <code>hal/riscv/</code> 和 <code>hal/loongarch/</code> 各自实现
最小化内核修改	只有7个文件需要 <code>#ifdef ARCH_loongarch</code> 条件编译，其余20+个内核文件完全平台无关
搬家不改逻辑	RISC-V HAL直接移植xv6原始源码，所有11个文件修改量<10%
翻译而非重写	LoongArch通过 <code>estat_to_scause()</code> 和 <code>sstatus_compat()</code> 将LoongArch CSR语义"翻译"为RISC-V语义， <code>kernel/trap.c</code> 无需修改
接口优先	先定义 <code>hal/</code> 下的7个接口头文件，再分别实现RISC-V和LoongArch版本
依赖倒转	<code>kernel/</code> 通过 <code>#include "hal/hal.h"</code> 依赖HAL，而不是HAL依赖内核

5.1.2 抽象子系统与接口对应

1	<code>kernel/</code>	
2	<code>main.c</code>	→ <code>hal_irq_init()</code> , <code>hal_irq_hart_init()</code> , <code>hal_timer_init()</code> , <code>virtio_disk_init()</code>
3	<code>trap.c</code>	→ <code>hal_read_scause()</code> , <code>hal_read_sepc()</code> , <code>hal_read_sstatus()</code> , <code>hal_irq_claim()</code>
4	<code>vm.c</code>	→ <code>hal_tlb_flush()</code> , <code>hal_vm_switch()</code> , <code>w_satp()/r_satp()</code>
5	<code>proc.c</code>	→ <code>hal_get_hartid()</code> , <code>hal_intr_on()</code> , <code>hal_intr_off()</code> , <code>hal_switch()</code>
6	<code>exec.c</code>	→ <code>hal_vm_map()</code> + ARCH conditional guard pages
7	<code>console.c</code>	→ <code>hal_console_init()</code> , <code>hal_putchar()</code> , <code>hal_getchar()</code>
8	<code>sysfile.c</code>	→ (不直接调用HAL，通过VFS层间接访问块设备)

5.1.3 HAL接口分层的设计考量

接口头文件	设计考量
<code>hal_arch.h</code>	必须处理两个架构CSR指令集完全不同 (<code>csrr/csrw</code> vs <code>csrrd/csrwr</code>)、特权级模型不同 (<code>M/S/U</code> vs <code>PLV0/PLV3</code>)
<code>hal_vm.h</code>	必须处理硬件自动walk (RISC-V Sv39) vs 软件TLB重填 (LoongArch 4级) 的最大差异
<code>hal_intr.h</code>	必须处理PLIC vs EIOINTC两种中断控制器的claim/complete协议差异
<code>hal_timer.h</code>	必须处理CLINT MMIO定时器 vs CSR定时器 (TCFG/TVAL) 的差异
<code>hal_ctx.h</code>	必须处理14 vs 12个callee-saved寄存器的上下文结构体差异 (通过条件编译)
<code>hal_console.h</code>	双输出路径 (同步轮询用于panic安全输出，中断驱动用于正常write())
<code>hal_memlayout.h</code>	各平台提供KERNBASE/PHYSTOP/UART基址等物理常量

5.1.4 与VFS的对比：两种抽象模式

维度	HAL（硬件抽象）	VFS（文件系统抽象）
抽象对象	CPU架构（RISC-V/LoongArch）	文件系统（xv6fs/ext2/FAT32）
接口机制	编译时选择（ARCH=riscv loongarch + 条件编译）	运行时注册（vfs_register + 虚函数表）
接口数量	~30个 hal_* 函数 + 7个接口头文件	13个 vnode_ops + 2个 vfs_ops
扩展方式	新增 hal/xxx/ 目录 + 实现所有接口	新增 xxx.c + 填充 vnode_ops 表 + 一行 vfs_register
修改影响面	7个内核文件需条件编译	0个内核文件需修改（VFS层完全隔离）

5.2 接口规范

头文件	抽象内容
hal_arch.h	CSR寄存器/CPU操作
hal_vm.h	页表操作
hal_intr.h	中断控制器
hal_timer.h	定时器
hal_memlayout.h	内存布局
hal_console.h	串口
hal_ctx.h	上下文切换

5.3 RISC-V实现

RISC-V平台是xv6的原生架构，HAL化的核心原则是"只搬家不改逻辑"——将平台相关代码从 kernel/ 移动到 hal/riscv/，添加 hal_* 前缀包装函数，不修改核心逻辑。

5.3.1 实现策略：薄包装（Thin Wrapper）

由于RISC-V是xv6的原生平台，所有CSR操作、异常处理、页表格式都是原生支持的。RISC-V HAL实现采用"搬家+改名"的策略：

原始文件	HAL文件	修改方式
kernel/riscv.h	hal/riscv/arch.h	CSR内联函数+PTE宏 + 末尾添加19个 hal_* 包装函数
kernel/entry.S	hal/riscv/hal_entry.S	仅改文件位置
kernel/start.c	hal/riscv/hal_start.c	M态初始化逻辑不变
kernel/trampoline.S	hal/riscv/hal_trampoline.S	用户态陷入/返回，保留sscratch机制
kernel/kernelvec.S	hal/riscv/hal_kvec.S	内核态中断向量，sret返回
kernel/swtch.S	hal/riscv/hal_swtch.S	14个callee-saved寄存器
kernel/plic.c	hal/riscv/hal_plic.c	PLIC中断控制器，添加 hal_irq_* 包装
kernel/uart.c	hal/riscv/hal_uart.c	16550a UART驱动，添加 hal_console_* 包装
kernel/virtio_disk.c	hal/riscv/hal_virtio.c	MMIO virtio-blk驱动，支持多磁盘(VIRTIO_NDISK)
kernel/kernel.ld	hal/riscv/kernel.ld	入口0x80000000，增加trampsec对齐
kernel/memlayout.h	hal/riscv/memlayout.h	QEMU virt物理地址布局

所有直接移植文件的修改量 < 10%：基本上只是"搬家+改include+加hal_*包装函数"，核心逻辑一行未改。

5.3.2 关键硬件特性

特性	RISC-V实现	说明
特权级	M/S/U三级	M态初始化(start.c)，S态运行内核，U态运行用户程序
CSR访问	csrr/csrw/csrwi/csrsi 指令	arch.h中约100个内联函数封装
页表	Sv39三级页表	PT_LEVELS=3, PXSHIFT=30/21/12, PTE格式含V/R/W/X/U/G/A/D标志位
TLB刷新	sfence.vma 指令	单条指令完成TLB全刷新
中断控制器	PLIC (Platform-Level Interrupt Controller)	MMIO基址0x0C000000，外设中断管理
定时器	CLINT + stimecmp CSR	内核态通过SBI或sstc扩展设置下一次中断
上下文切换	14个callee-saved寄存器(ra,sp,s0-s11)	hal_swtch.S保存/恢复
用户态陷入	sscratch寄存器保存trapframe指针	硬件自动交换，uservec入口
virtio 磁盘	MMIO方式，基址 0x10001000(disk0)/0x10002000(disk1)/0x10003000(disk2)	支持最多3个磁盘

5.3.3 启动流程

```
1 QEMU加载内核 → hal_entry.S(读mhartid,设栈) → hal_start.c(start)
2   → M态配置(PMP,中断委托,mideleg/medeleg)
3   → 定时器初始化
4   → mret切换到S态 → kernel/main.c(main)
5   → kinit() 物理内存初始化
6   → kvm_init() 内核页表创建(Sv39)
7   → kvminithart() 写satp使能MMU
8   → procinit() 进程表初始化
9   → trapinit/trapinithart() 陷阱向量设置
10  → hal_irq_init/hal_irq_hart_init() PLIC初始化
11  → virtio_disk_init() 块设备初始化
12  → userinit() 第一个用户进程
13  → scheduler() 调度循环
```

5.3.4 为FAT32新增的扩展

为支持FAT32文件系统，RISC-V HAL做了以下扩展（不影响xv6fs和ext2）：

修改文件	新增内容	说明
hal/riscv/memlayout.h	VIRTIO2=0x10003000, VIRTIO2_IRQ=3	第三个virtio磁盘的MMIO基址和中断号
hal/riscv/hal_virtio.c	VIRTIO2加入 virtio_base[] 数组，NDISK上限扩展为3	支持三个virtio磁盘
hal/riscv/hal_plic.c	NDISK>2时启用VIRTIO2_IRQ(irq=3)	PLIC中断使能
kernel/vm.c	NDISK>2时映射VIRTIO2的PGSIZE页表	内核地址空间映射
kernel/trap.c	NDISK>2时处理VIRTIO2_IRQ中断	devintr()中断分发

以上修改均通过 `#if VIRTIO_NDISK > 2` 条件编译保护，不影响LoongArch(NDISK=1)和RISC-V原有双磁盘配置。

5.4 LoongArch移植

LoongArch(LoongArch 64)是龙芯中科自主设计的RISC架构，与RISC-V存在根本性差异。本移植使xv6以完整功能运行在QEMU LoongArch virt机器上，所有usertests通过。

5.4.1 架构核心差异

维度	RISC-V (rv64)	LoongArch (LA64)	HAL实现影响
特权级	M/S/U三级	PLV0(内核)/PLV3(用户)两级	无需M态启动, start()直接运行在PLV0
CSR指令	<code>csrr/csrw/csric/csrsi</code>	<code>csrrd/csrwr/csrxchg</code>	全部CSR内联函数重写
页表walk	硬件自动(Sv39, 3级)	软件TLB重填 (4级页表)	最大差异! 需实现TLBREENTRY处理函数
TLB刷新	<code>sfence.vma</code>	<code>invtlb 0, \$r0, \$r0</code>	指令不同但语义等价
异常返回	<code>sret/mret</code>	<code>ertn</code> (统一)	所有异常返回用 <code>ertn</code>
中断控制器	PLIC (MMIO)	EIOINTC (MMIO)	hal_intr.c完全重写
定时器	CLINT MMIO + stimecmp CSR	TCFG/TVAL CSR (纯CSR)	hal_timer接口重写
virtio磁盘	MMIO virtio @0x10001000	无virtio→改用 ramdisk	hal_virtio.cl以objcopy嵌入镜像
Callee-saved	ra,sp,s0-s11 (14个)	ra,sp,fp,s0-s8 (12个)	hal_ctx.h条件编译, hal_swtdch.S寄存器数不同
PTE格式	PPN连续[53:10]	PPN分两段 [47:0]+[52:48]	PA2PTE/PTE2PA宏完全重写
页表级数	3级 (9-9-9-12)	4级 (10-10-10-12)	PT_LEVELS=4, PXSHIFT宏不同
用户程序链接	从0x0开始	从0x2000开始	跳过低2页保护页

5.4.2 LoongArch HAL文件清单

#	文件	行数	职能
1	hal/loongarch/arch.h	462	CSR号定义、CRMD/PRMD位域、ECFG/ESTAT位域、异常码、CSR读写内联、estat_to_scause()转换、RISC-V兼容别名
2	hal/loongarch/memlayout.h	66	QEMU virt物理地址布局：UART0=0x1FE001E0, RAM基址, flash/RAM分离
3	hal/loongarch/hal_entry.S	18	启动入口：读CUID CSR → 设栈 → 跳start()
4	hal/loongarch/hal_start.c	89	flash→RAM搬运、多核同步、DMW直接映射窗口配置、定时器初始化
5	hal/loongarch/hal_trampoline.S	140	用户态陷入/返回：uservec(KSave0/1保存trapframe指针→切页表→跳usertrap)+userret(恢复寄存器→ertn)
6	hal/loongarch/hal_kvec.S	63	内核态中断向量：保存17个caller-saved寄存器→kerneltrap()→恢复→ertn
7	hal/loongarch/hal_swthc.S	41	上下文切换：保存/恢复12个callee-saved寄存器(ra,sp,fp,s0-s8)
8	hal/loongarch/hal_tlbrefill.S	72	TLB重填处理器(TLBREENTRY)：lddir/ldpte 4级walk + P/W清除 + MAXVA检查 + tlbfill
9	hal/loongarch/hal_intr.c	90	EIOINTC中断控制器：init/hart_init/claim/complete
10	hal/loongarch/hal_uart.c	106	16550a UART驱动(QEMU兼容)
11	hal/loongarch/hal_virtio.c	130	RAM-disk块设备：objcopy嵌入镜像→kalloc可写页→memmove复制→bread/bwrite服务
12	hal/loongarch/kernel.ld	67	链接脚本：flash/RAM分离布局，入口0x1C000000
13	hal/loongarch/user.ld	39	用户程序链接脚本：入口0x2000(跳过保护页)

5.4.3 关键设计决策

① estat_to_scause() — 异常码转换层

LoongArch与RISC-V的异常码完全不同。例如LoongArch的系统调用ecode=11，而RISC-V的scause=8。为了让kernel/trap.c无需修改，在arch.h中实现了estat_to_scause()转换函数：

LoongArch Ecode	名称	RISC-V scause
11 (SYS)	系统调用	8
1 (PIL)	load缺页	13
2 (PIS)	store缺页	15
3 (PIF)	fetch缺页	12
13 (INE)	非法指令	2
8 (ADE)	地址错误	12或13(依赖EsubCode)

关键陷阱：LoongArch的ADE(8)恰好与RISC-V的syscall scause(8)冲突，必须用EsubCode区分——ADE(0)→12，ADE(1)→13。

② 软件TLB重填 (hal_tlbrefill.S)

LoongArch MMU在TLB miss时不自动walk页表，而是触发TLBREENTRY异常。内核必须通过软件完成页表walk和TLB填入。本实现使用4级页表walk(lddir/ldpte指令)，逐级检查PTE有效位后执行tlbfill填入TLB。

同时，内核空间使用DMW(Direct Mapping Window)直接映射，避免内核TLB miss。用户态MAXVA保持xv6原始契约 $1 \ll 38$ 。

③ RAM-disk替代virtio

LoongArch QEMU virt机器的块设备通过PCI virtio而非MMIO暴露。为避免实现完整的PCI枚举和virtio-over-PCI驱动，本移植采用ramdisk方案：通过objcopy将fs.img嵌入内核二进制，在内存中分配可写页面并复制镜像数据，bread/bwrite直接操作内存。

④ RISC-V兼容别名

LoongArch arch.h末尾定义了RISC-V CSR函数名的#define 别名（如w_satp→w_pgd1, w_stvec→w_eentry），使得内核通用代码中对RISC-V CSR的遗留调用自动映射到LoongArch实现，减少对内核层的修改。

5.4.4 验证状态

- ☒ make ARCH=loongarch qemu 正常启动，进入shell
- ☒ usertests -q 全部测试通过
- ☒ 双磁盘编译(VIRTIO_NDISK=2)通过
- ☒ ext2文件系统基本测例通过(ext2test1, ext2test2)
- ☒ FAT32不支持（受QEMU ROM限制，见第六章）
- ☒ ext2test3 L4并发测试已知问题(同事侧)

5.5 代码量统计

5.5.1 HAL接口层（平台无关）

文件	行数	说明
hal/hal.h	30	统一入口，ARCH条件包含+聚合子系统头文件
hal/hal_arch.h	38	CPU控制接口（CSR读写、核心ID、中断使能）
hal/hal_intr.h	14	中断控制器接口
hal/hal_timer.h	17	定时器接口
hal/hal_vm.h	18	虚拟内存接口（TLB刷新+PTE常量）
hal/hal_memlayout.h	17	内存布局抽象
hal/hal_console.h	14	控制台I/O接口
hal/hal_ctx.h	49	上下文切换接口（ARCH条件结构体）
小计	~197	8个接口头文件

5.5.2 RISC-V平台实现

文件	行数	说明
hal/riscv/arch.h	394	CSR内联函数+PTE宏(Sv39)+19个hal_*包装
hal/riscv/memlayout.h	66	QEMU virt物理地址布局
hal/riscv/hal_entry.S	21	启动入口
hal/riscv/hal_start.c	77	M态初始化
hal/riscv/hal_plic.c	67	PLIC中断控制器
hal/riscv/hal_uart.c	165	16550a UART驱动
hal/riscv/hal_virtio.c	375	Virtio-blk MMIO驱动(多磁盘)
hal/riscv/hal_swtdh.S	42	上下文切换(14寄存器)
hal/riscv/hal_trampoline.S	149	用户态陷入/返回
hal/riscv/hal_kvec.S	62	内核态中断向量
hal/riscv/kernel.ld	45	链接脚本
小计	~1463	11个实现文件

5.5.3 LoongArch平台实现

文件	行数	说明
hal/loongarch/arch.h	462	CSR内联+PTE宏(4级)+estat_to_scause+兼容别名
hal/loongarch/memlayout.h	66	QEMU virt物理地址布局
hal/loongarch/hal_entry.S	18	启动入口
hal/loongarch/hal_start.c	89	flash→RAM搬运+多核同步
hal/loongarch/hal_trampoline.S	140	用户态陷入/返回(KSave0/1机制)
hal/loongarch/hal_kvec.S	63	内核态中断向量(17寄存器)
hal/loongarch/hal_swtdh.S	41	上下文切换(12寄存器)
hal/loongarch/hal_tlbrefill.S	72	TLB重填处理器(lddir/ldpte walk)
hal/loongarch/hal_intr.c	90	EIOINTC中断控制器
hal/loongarch/hal_uart.c	106	16550a UART驱动
hal/loongarch/hal_virtio.c	130	RAM-disk块设备
hal/loongarch/kernel.ld	67	链接脚本(flash/RAM分离)
hal/loongarch/user.ld	39	用户程序链接(0x2000起始)
小计	~1383	13个实现文件

5.5.4 HAL总计

类别	文件数	行数
HAL接口头文件	8	~197
RISC-V实现	11	~1463
LoongArch实现	13	~1383
HAL合计	32	~3043

5.5.5 ARCH条件编译分布

整个项目中 `#ifdef ARCH_loongarch` 仅出现在**7个文件**中：

文件	位置	用途
<code>hal/hal.h</code>	包含路径选择	条件包含arch.h和memlayout.h
<code>hal/hal_ctx.h</code>	struct hal_context	12字段 vs 14字段
<code>kernel/vm.c</code>	5处	kvmmake, MMU使能, freewalk, tlb_fill
<code>kernel/proc.c</code>	2处	跳过TRAMPOLINE/TRAPFRAME映射
<code>kernel/exec.c</code>	1处	低2页PLV0保护页映射
<code>kernel/trap.c</code>	1处	KSave1存储陷阱帧内核VA
<code>Makefile</code>	多处	工具链、QEMU选项、用户链接

关键设计原则：**只有7个文件需要ARCH条件编译，其余20+个内核文件完全平台无关。**

第六章 FAT32文件系统适配

说明：由于开发时间紧张（赛前两天），FAT32文件系统的实现使用了AI辅助开发工具（详见附录D）。目前仅完成了RISC-V平台的适配，LoongArch平台因QEMU ROM大小限制未能实现。

6.1 FAT32简介

FAT32（File Allocation Table 32）是微软在FAT16基础上扩展的文件系统，广泛用于U盘、SD卡等可移动存储介质。其特点是结构简单、兼容性好。FAT32的关键数据结构包括：BPB（BIOS Parameter Block，位于扇区0）、FAT表（每个表项32-bit，有效28-bit）、32字节的8.3短目录项。

本实现将FAT32作为xv6 VFS的第三个文件系统后端，通过 `vfs_register(fat32_mount, "fat32")` 注册，遵循与ext2相同的注册模式。

6.2 文件结构

文件	行数	说明
<code>kernel/fat32.h</code>	76	数据结构： <code>fat32_bpb</code> (BPB), <code>fat32_dirent</code> (32字节目录项), FAT常量
<code>kernel/fat32.c</code>	301	VFS后端实现： mount/lookup/read/write/create/unlink/mkdir/readdir/stat/truncate
<code>user/fat32test.c</code>	81	6项自动化测试： mount/create/write/lookup+stat/mkdir/nested/unlink
合计	458	

6.3 适配策略

策略	说明
仅RISC-V	LoongArch因QEMU ROM限制未适配（见6.7节）
独立virtio磁盘	FAT32使用第三个virtio磁盘(dev=3)，10MB独立镜像
512字节扇区适配	xv6使用BSIZE=1024块，FAT32使用512字节扇区，通过sread()/swrite()辅助函数做扇区→块映射
8.3短文件名only	不实现LFN长文件名，文件名使用11字节空间填充格式
单簇=1扇区	BPB_SecPerClus=1，简化簇分配逻辑
双FAT同步更新	每次FAT写入同时更新两份FAT表

6.4 磁盘布局与核心计算

6.4.1 FAT32镜像布局

1	[扇区 0: BPB + 启动代码 (512B)]
2	[扇区 1: FSInfo扇区]
3	[扇区 2-31: 保留区]
4	[FAT表1: BPB_FATSz32 扇区]
5	[FAT表2: BPB_FATSz32 扇区]
6	[数据区: 簇2 = 根目录开始]

6.4.2 核心计算公式

1	FirstDataSector = BPB_RsvdSecCnt + BPB_NumFATS × BPB_FATSz32
2	FirstSectorOfCluster(N) = ((N - 2) × BPB_SecPerClus) + FirstDataSector
3	FAT表项偏移 = N × 4 (每项4字节)
4	FAT表项所在扇区 = BPB_RsvdSecCnt + (偏移 / BPB_BytsPerSec)

6.4.3 512字节扇区→1024字节块的转换

这是本次实现中遇到的核心技术难点。xv6的 bread() / bwrite() 使用固定块大小 BSIZE=1024，而FAT32标准使用 BPB_BytsPerSec=512 扇区。每1个xv6块 = 2个FAT32扇区。

1	xv6块0: [扇区0 (512B)][扇区1 (512B)]
2	xv6块1: [扇区2] [扇区3]

所有FAT32磁盘I/O必须通过sread()/swrite()辅助函数，封装扇区→块的转换：

```
1 static int sread(struct fat32_mount *fm, uint sec, uint off, void *buf, uint n) {
2     uint blk = sec / fm->spb; // 扇区号→块号
3     uint bo = (sec % fm->spb) * fm->bps + off; // 块内偏移
4     struct buf *bp = bread(fm->dev, blk);
5     if (!bp) return -1;
6     memmove(buf, bp->data + bo, n);
7     brelse(bp);
8     return 0;
9 }
```

其中 spb = BSIZE / bytes_per_sec (通常1024/512=2)，bps = BPB_BytsPerSec (512)。

6.5 数据结构

6.5.1 BPB (BIOS Parameter Block)

位于FAT32镜像的第0扇区。关键字段：

字段	偏移	大小	说明
BPB_BytsPerSec	11	2	每扇区字节数，必须为512
BPB_SecPerClus	13	1	每簇扇区数
BPB_RsvdSecCnt	14	2	保留扇区数，FAT32典型值32
BPB_NumFATs	16	1	FAT表份数，通常为2
BPB_TotSec16/32	19/32	2/4	总扇区数（≤32MB时TotSec16有效）
BPB_FATSz32	36	4	FAT32特有：每个FAT表的扇区数
BPB_RootClus	44	4	根目录起始簇号，通常为2

6.5.2 目录项 (32字节)

1	Offset	Size	Field
2	0	11	DIR_Name（8.3短文件名，空格填充，首字节0xE5=已删除）
3	11	1	DIR_Attr（ATTR_DIRECTORY=0x10，ATTR_ARCHIVE=0x20）
4	20	2	DIR_FstClusHI（起始簇号高16位）
5	26	2	DIR_FstClusLO（起始簇号低16位）
6	28	4	DIR_FileSize（文件大小，uint32）

6.5.3 FAT表项

FAT32每个表项32-bit（有效28-bit，高4位保留）：

值	含义
0x00000000	空闲簇
0x0FFFFFF7	坏簇
>= 0x0FFFFFF8	簇链结束(EOC)

6.6 已实现的VFS操作

vnode_ops	实现函数	实现要点
lookup	flookup	扫描目录项(32字节为单位)，按8.3名匹配（大写转换+空格padding）
read	fread	遍历FAT簇链，通过sread()逐扇区读取数据
write	fwrite	写入数据，必要时alloc_clus扩展簇链，通过swrite()写回
create	fcreate	分配目录项(dir_add) + 目录型创建"."和".." + alloc_clus分配首簇
unlink	funlink	标记目录项DIR_Name[0]=0xE5 + free_chain释放FAT簇链 + 目录需检查为空
mkdir	fmkdir	委托fcreate(V_DIR)，父目录links_count++
readdir	freaddir	遍历目录项，跳过已删除(0xE5)和LFN项(ATTR_LONG_NAME)，返回vdirent
stat	fstat	返回文件类型(V_FILE/V_DIR)和大小
truncate	ftrunc	free_chain释放簇链+重置file_size=0

核心helper函数：

- clus_to_sector(N) — 簇号→数据区首个扇区号
- read_fat(N) / write_fat(N, val) — 读写FAT表项（含双FAT同步更新）
- alloc_clus() — 线性扫描FAT表找空闲簇(值==0)
- free_chain(start) — 遍历簇链逐项清零

6.7 LoongArch 未适配说明

LoongArch平台未能支持FAT32文件系统。

根本原因：LoongArch QEMU virt机器使用 -bios 方式加载raw binary内核，ROM加载上限约**4MB**。当前LoongArch内核体积为：

```
1 fs.img = xv6(2MB) + ext2(1MB) = 3MB → kernel.bin ≈ 3.2MB
2 加 fat32.img = 1MB → kernel.bin ≈ 4.2MB → 超过4MB ROM限制
```

深层原因：LoongArch HAL层没有实现PCI virtio磁盘驱动，而使用ramdisk方案（通过 objcopy 将磁盘镜像嵌入内核二进制）。RISC-V拥有MMIO virtio驱动，可通过QEMU -drive 选项动态挂载任意大小的磁盘镜像，不受ROM限制。

可能的解决方案（均因赛前时间限制未实施）：

- 为LoongArch实现PCI virtio磁盘驱动 —— 工作量较大（需PCI枚举+virtio-over-PCI协议）
- 压缩磁盘镜像（ext2减至512KB，FAT32减至512KB） —— 可能影响ext2测试文件空间
- 探索QEMU的其他内核加载方式（-kernel + initrd） —— QEMU LoongArch virt目前不支持

结论：在现有ramdisk架构下，LoongArch FAT32暂不可行。保留RISC-V only。

6.8 测试验证

6.8.1 测试环境

在RISC-V Docker容器中执行：

```
1 cd /xv6/xv6-riscv-xv6-riscv-rev5
2 make clean && make qemu
```

启动后在xv6 shell中输入 fat32test 运行自动化测试。

6.8.2 测试结果

```
1 $ fat32test
2 fat32test: starting
3 PASS: mount
4 PASS: create + write
5 PASS: lookup + stat
6 PASS: mkdir
7 PASS: create nested
8 PASS: unlink
9 fat32test: ALL PASSED
```

6项测试全部通过，覆盖：挂载(dev=3)、创建文件+写入数据、重新打开+获取属性、创建子目录、嵌套路径创建文件、删除文件并验证不可访问。

详细测试用例设计见 fat32-design.md。

6.9 mount失败的三个关键问题与解决方案

问题1: 512字节扇区 vs 1024字节块

现象：mount /fat 3 fat32 失败，BPB解析不正确。

根因：xv6的 bread() / bwrite() 使用 BSIZE=1024 固定块大小。直接用FAT32扇区号调用 bread() 会导致位置错误——扇区0和扇区1被当作同一个1024字节块读取。

解决方案：引入 sread() / swrite() 辅助函数，自动完成 blkno=sec/spb; offset=(sec%spb)*bps 的转换。

问题2: BPB_TotSec16不为0

现象：以 BPB_TotSec16 != 0 作为非FAT32的判断条件，导致小镜像mount被拒绝。

根因：FAT32规范要求 TotSec16=0（使用 TotSec32），但Linux的 mkfs.fat 对≤32MB的小镜像将总扇区数填入 TotSec16（16位够用），TotSec32 保持为0。

解决方案：使用 TotSec16 ? TotSec16 : TotSec32 作为总扇区数，FAT32类型判定仅依赖 BPB_FATSz32 != 0。

问题3: xv6fs inode耗尽

现象：mkdir /fat 在shell中手动执行失败。

根因：xv6fs使用固定大小的inode表(13 blocks × 16 inodes/block = 208 inodes)，用户程序消耗大量inode。

解决方案：在 init.c 启动时预先创建 /fat 目录（与 /mnt 相同方式处理）。

6.10 已知限制

限制	说明
仅RISC-V	LoongArch不支持（ROM大小限制）
仅8.3短文件名	无LFN长文件名支持
无时间戳	不维护atime/mtime/ctime
单簇=1扇区	未实现多扇区簇
无子目录深度限制检查	理论可创建任意深度目录
10MB镜像上限	mkfs.fat对10MB以下镜像有警告(不影响使用)
无并发保护	多核并发FAT I/O未加锁（xv6测试场景为单核）

第七章 测试与验证

7.1 测试环境

项目	版本/配置
QEMU (RISC-V)	9.0.0
QEMU (LoongArch)	——
编译器	——
操作系统	Linux (Ubuntu/Debian)

7.2 xv6原生测例

测试	RISC-V	LoongArch
usertests	☐	☐
forktest	☐	☐
grind	☐	☐

7.3 赛方测试用例

测试	来源	RISC-V 结果
ext2test1	赛方	☐
ext2test2	赛方	☐

7.4 自定义综合测试（ext2test3）

RISC-V 当前结果（HAL分支）： 13 passed, 2 failed

层次	测试项	RISC-V 结果
L1.1	mount→umount→remount 循环	PASS
L1.2	mount 错误路径	PASS
L2.1	create + lookup 嵌套	PASS
L2.2	跨块读写 4096 字节	PASS
L2.3	readdir 验证	PASS
L2.4	mkdir + type 检查	PASS
L2.5	unlink 语义	PASS
L2.6	硬链接 nlink 验证	PASS
L2.7	mknod 设备节点	PASS
L2.8	stat 完整性	PASS
L2.9	truncate (O_TRUNC)	PASS
L3	路径遍历 (5 子项)	PASS
L4.1	并发 write (fork)	FAIL
L4.2	unlink-while-open	PASS
L4.3	大文件 200 块 (间接块)	FAIL

7.5 赛题参考测试结果

```
1  $ cd tests
2  $ cat cmds.sh | sh
3  ----- test1: begin... -----{
4  test_copy_file(/mnt/hello.c,/hello.c): succeed
5  test_copy_file(/mnt/hello.c,/tests/hello.c): succeed
6  test_copy_file(/hello.c,/mnt/hello.c): succeed
7  test_copy_file(/tests/hello.c,/mnt/hello.c): succeed
8  ----- test1: finished -----}
9
10 ----- test2: begin... -----{
11  将从 xv6 文件系统复制文件到 EXT2 文件系统
12  文件复制成功!
13  验证复制结果:
14  复制成功! 文件内容:
15  #include "kernel/types.h"
16  #include "user/user.h"
17  void main () { printf ("Hello, world!\\n"); }
18  test_cross_fs_copy(): succeed
19  ----- test2: finished -----}
```

7.6 FAT32测试

测试程序: user/fat32test.c (81行) , 6项自动化测试

RISC-V测试结果:

测试项	操作	结果
mount	<code>mount("/fat", 3, "fat32")</code>	PASS
create + write	<code>open+write("/fat/hello.txt", 16 bytes)</code>	PASS
lookup + stat	<code>open+open("/fat/hello.txt", O_RDONLY)</code>	PASS
mkdir	<code>mkdir("/fat/subdir")</code>	PASS
create nested	<code>open+write("/fat/subdir/nested.txt", 6 bytes)</code>	PASS
unlink	<code>unlink("/fat/hello.txt")</code> + 确认不可再打开	PASS

LoongArch: 不支持 (无FAT32镜像可用)

7.7 已知问题

测试项	症状	初步分析
L4.1 — concurrent write	<code>first half mixed</code> (test_l4_concurrent:515)	多进程并发写入同一文件时，vnode锁或日志配额可能导致写入交错。需排查 ext2_write 的锁粒度和 begin_op/end_op 包裹范围
L4.3 — large file (indirect)	<code>create l4big failed: fd=-1</code> (test_l4_large_file:569)	大文件（200块，需使用单间接块）创建失败。可能原因：① 间接块分配/清零逻辑有 bug；② balloc 空间不足（镜像偏小）；③ bmap alloc=1 路径对间接块的处理有误
LoongArch 适配	已解决	早期 <code>merge-vfs-hal</code> 分支曾探索多磁盘方案（每FS独立磁盘），该方案在LoongArch HAL层缺乏多virtio设备支持。final-submission分支改为单磁盘分区方案（xv6fs与ext2共享 fs.img，通过EBLK宏偏移），避免此问题

第八章 设计精巧度分析

8.1 与Linux VFS对比

维度	Linux	本作品
VFS代码量	>10万行	~664行 (vfs.c + vfs.h)
支持FS类型	数十种	3种 (xv6fs + ext2 + FAT32)
核心抽象	多层继承 (dentry/inode/address_space/...)	统一vnode+ops
设计目标	高性能通用	教学精简

8.2 整体代码量统计

类别	文件	行数
VFS核心	kernel/vfs.c + kernel/vfs.h	553 + 111 = 664行
ext2	kernel/ext2.c + kernel/ext2.h	1037 + 128 = 1165行
xv6fs	kernel/xv6fs.c	499行
上层适配	kernel/sysfile.c + kernel/file.c	386 + 163 = 549行
FAT32	fat32.c/h	377行
HAL	hal/ (RISC-V + LoongArch + 接口层)	3043行
合计新增		~4340行
原xv6-rev5		~10000行
增幅		~43%

8.3 设计模式

模式	应用位置	效果
策略模式	vnode_ops	每FS独立实现，VFS层不感知
注册表模式	fstypes[]	添加新FS仅需一行注册
句柄模式	vnode	所有文件操作统一入口

8.4 与参考项目对比

维度	本作品	静春山 (SpringOS)	RuOK
VFS代码行数	664行	~200行 (薄适配层)	?
ext2/4代码行数	1165行 (ext2手写)	~4万行 (集成lwext4库)	?
支持FS数量	3	1 (仅ext4)	?
独立HAL	是	是	?
设计路线	从零手写，教学精简	集成成熟库，功能完整	?

第九章 挑战与展望

9.1 技术难点回顾

- ext2 orphaned延迟释放的并发安全
- "."跨文件系统返回
- mount失败路径的完整资源清理
- VFS路径穿越的性能优化
- ext2合并镜像偏移方案 (EBLK宏)：单磁盘双分区，避免多virtio设备在LoongArch上的兼容问题

9.2 未来工作

- 修复 ext2 L4.1 concurrent write 和 L4.3 large file 测试
- LoongArch HAL 的PCI virtio磁盘驱动实现 (解决FAT32/LoongArch ROM限制问题)
- FAT32 长文件名(LFN)支持
- 更多文件系统 (tmpfs、procfs)
- VFS写缓存与预读
- FAT32并发安全保护

9.3 收获与体会

本项目通过从零设计并实现硬件抽象层(HAL)和虚拟文件系统(VFS)，在真实的xv6教学操作系统上实践了操作系统的两大核心抽象——跨平台硬件虚拟化和多文件系统统一访问。主要收获：

1. **架构设计的价值**：HAL的三明治架构和VFS的vnode_ops模式证明了"先设计接口，再分别实现"是控制复杂度的有效方法。添加FAT32仅需一行 `vfs_register`，正是这种设计的直接收益。
2. **512字节扇区陷阱**：在FAT32实现中，xv6固定1024字节块与FAT32标准512字节扇区的不匹配是一个容易被忽视但影响深远的问题。这提示我们：即使是最简单的假设（块大小=扇区大小），在不同文件系统间也不能视为理所当然。
3. **LoongArch QEMU ROM限制**：4MB的ROM上限是前期未预料到的硬件约束。这一经验提醒我们：在嵌入式/教学OS中，内核镜像体积可能成为架构适配的硬瓶颈。ramdisk方案简化了块设备实现，但也以牺牲大镜像支持为代价。
4. **AI辅助开发的边界**：在时间紧迫的赛前冲刺中，AI工具显著加快了代码生成和文档编写速度（FAT32后端~380行代码一次生成），但关键设计决策（如512字节扇区映射）和硬件约束验证（ROM大小）必须由开发者主导。

附录

A. 编译与运行

```
1  # 编译
2  make
3
4  # 制作ext2和FAT32镜像
5  make ext2.img
6  make fat32.img
7
8  # 运行
9  make qemu
10
11 # ext2测试
12 mount /mnt 1 ext2
13 ext2test1
14 ext2test2
15 ext2test3
16
17 # FAT32测试
18 fat32test
```

B. O_APPEND 与 O_TRUNC 支持

<!-- TODO: 在 file.c/sys_open 中的处理

- O_TRUNC: vfs_open 路径中，若标志位包含 O_TRUNC，调用 ops->truncate
 - O_APPEND: file_write 中，将偏移量设为文件末尾再写入
 - 标志位定义见 kernel/fcntl.h
- >

C. 参考资料

- xv6-riscv-rev5: <https://github.com/mit-pdos/xv6-riscv>
- 赛题测试仓库: <https://github.com/yanjun-wen/xv6-extend-vfs>
- 参考项目1(静春山/SpringOS): <https://gitlab.eduxiji.net/educg-group-36002-2710490/T202510558995330-264>
- 参考项目2(RuOK): <https://gitlab.eduxiji.net/educg-group-36002-2710490/T202510486995232-2402>

D. 使用AI协助开发

本项目在开发过程中使用了AI辅助编程工具（Claude Code框架 + DeepSeek V4 API）。

| 项目 | 说明 |
|------|--------------------------------|
| AI框架 | Claude Code (Anthropic官方CLI工具) |
| 底层模型 | DeepSeek V4 API |
| 使用模式 | 交互式对话进行方案讨论、代码生成、编译调试、文档编写 |
| 工作目录 | xv6-riscv-xv6-riscv-rev5/ |

D.1 HAL开发日志：CLAUDE.md — 开发者主导、AI辅助记录

CLAUDE.md 是本项目HAL开发的核心指导文档，由HAL开发者（xiao-yang-lab）与AI共同书写，贯穿整个开发周期。所有架构设计决策由开发者做出，AI负责将决策系统化记录并辅助代码实现。

D.1.1 CLAUDE.md中的关键决策记录

| 决策 | 考量 |
|----------------------|--|
| 三明治架构 | 顶层kernel只调hal*接口 → 中层hal/hal*.h统一声明 → 底层hal/riscv/和hal/loongarch/各自实现 |
| "搬家不改逻辑" | RISC-V是xv6原生平台，HAL化不需要重构，只需移动文件+改include+加包装函数。AI扫描xv6源码后标注每个文件的修改量(<10%)，确认搬运可行性 |
| HAL接口切分为7个头文件 | 按功能子系统独立（arch/vm/intr/timer/ctx/console/memlayout），避免单一巨型头文件 |
| LoongArch使用RAM-disk | 避免PCI virtio枚举的复杂度，用objcopy嵌入镜像方案。AI分析了两种块设备方案的代码量和风险 |
| estat_to_scause() 转换 | LoongArch异常码与RISC-V完全不同，必须显式转换才能让trap.c共享。AI对照两架构手册提出映射表，开发者逐一验证正确性（特别是ADE=8与syscall=8的冲突处理） |
| #define兼容别名 | w_satp-w_pgd1 等宏让kernel/vm.c中残留的RISC-V CSR调用在LoongArch编译时自动重定向，减少条件编译 |
| MAXVA保持1<<38 | 虽然LoongArch硬件支持48-bit VA，但为保持xv6原始地址契约不变。开发者在hal_tlbrefill.S中落实边界检查代码 |

D.1.2 CLAUDE.md指导下的开发流程

CLAUDE.md记录了从第1周到第8周的完整开发过程：第1周源码分析和平台代码标注→第2周在RISC-V上建立HAL框架→第3周完成全部接口→第4周搭建LoongArch Docker环境→第5-6周LoongArch HAL核心实现→第7周整合调试→第8周收尾。每周的交付物、验证标准和实际完成情况均在CLAUDE.md中记录。

CLAUDE.md由开发者和AI共同维护：在每个阶段开始时更新目标和要求，AI在每次交互后更新实际进展和Bug修复记录。这份文档确保了跨Session的项目上下文不丢失。

D.2 HAL核心代码调试：AI的局限与开发者的改进

D.2.1 TLB重填处理器：AI多轮迭代未能解决，开发者找到最终方案

TLB重填（hal/loongarch/hal_tlbrefill.S，72行）是整个LoongArch移植中最复杂、调试时间最长的代码。LoongArch的MMU在TLB miss时不像RISC-V那样硬件自动walk页表，而是触发TLBREENTRY异常，由内核软件完成页表walk和TLB填入。

AI在理解这个机制时经历了多轮失败迭代：

| 迭代 | AI的方案 | 问题 | 最终解决 |
|-----|---|--|--|
| 第1轮 | AI试图用C语言在vm.c中实现TLB walk | TLB refill handler运行在DA=1,PG=0的直接地址翻译模式下，不能使用内核栈，C函数调用会破坏寄存器上下文 | 改用纯汇编在hal_tlbrefill.S中实现，handler只能使用TLBRSAVE这一个CSR保存临时寄存器 |
| 第2轮 | AI尝试用KSave0/KSave1多个CSR保存中间基址，避免从PGDL重走 | KSave0/1已被uservec用于保存用户a0和trapframe指针，不可复用 | handler只剩\$t0一个临时寄存器可用，改为"每级从PGDL重走"的方案 |
| 第3轮 | AI写出了4级lddir walk的基本框架 | 没有处理lddir返回值的V标志——lddir成功时返回{PPN_base, 1}（bit0=V=1），直接作为下一级基址会访问错误地址 | 使用addi.d -1清除V标志：物理页基址页对齐（低12位=0），bit0=0，{PPN_base,1}-1=PPN_base，无借位 |
| 第4轮 | AI没有设置TLBREHI.PS字段 | TLBREHI.PS未初始化→tlbfill可能将条目填入MTLB（大页槽）→后续4KB页查找无法命中→无限TLB miss循环 | 在入口处添加PS=12的设置：srli.d+slli.d清低6位+ori设置4KB页大小 |
| 第5轮 | AI的异常路径直接ertn | 如果handler直接ertn而不填TLB，CPU重试访问→再次TLB miss→再次进入handler→无限循环 | 添加.Ltlbr_invalid路径：填一个V=0的无效条目到TLB，重试时触发TLB hit(V=0)→标准缺页异常→进入usertrap→vmfault处理 |

最终版的hal_tlbrefill.S执行序列：保存\$t0→设PS=12→MAXVA边界检查→4级lddir walk（每级检查V后重走→addi.d -1清V）→ldpte→清除P/W保留位→tlbfill→ertn。这段代码是整个HAL中投入精力最多的部分。

D.2.2 AI的其他局限与开发者的纠正

| AI的局限 | 开发者的纠正 |
|---|--|
| AI建议LoongArch MAXVA放宽到1<<46以利用硬件能力 | 予以驳回：必须保持xv6原始用户地址契约（MAXVA=1<<38），否则usertests中的MAXVAplus等边界测试语义被破坏 |
| AI在hal_start.c中遗漏了多核同步逻辑 | 补充：Core 0完成flash→RAM搬运后设置READY_FLAG="la64done"的hex值，其他核自旋等待 |
| AI对PTE PPN分两段（PTE[59:12]和PTE[11:7]）的PA2PTE/PTE2PA宏使用了复杂的位操作 | 简化为单掩码方案：因为QEMU virt的物理地址不超过256MB，PPN高位始终为0，用 &0xFFFFFFFFFFFF000 单掩码即可 |
| AI初期未正确处理exec低2页保护（LoongArch TLB lookup中高VA可能别名到低VPPN） | 添加低2页PLV0保护页映射 + user.ld从0x2000链接 |

D.3 FAT32实现：AI生成代码框架、开发者发现关键缺陷

FAT32文件系统的实现由队友陈权负责。由于赛前时间紧张（仅两天），FAT32的代码框架由AI辅助生成（数据结构定义、VFS后端操作函数、测试程序），但**最关键的bug是由HAL开发者（xiao-yang-lab）发现并指出的。**

D.3.1 512字节扇区映射问题：HAL开发者的发现

AI生成的初版 fat32.c 直接将FAT32扇区号传给xv6的 bread() 函数。在RISC-V测试环境中，mount /fat 3 fat32 失败——读取的BPB数据全部错位。

根因分析：xv6使用 BSIZE=1024 固定块大小，FAT32标准使用512字节扇区。每1个xv6块 = 2个FAT32扇区。直接将FAT32扇区号用作 bread() 的块号参数，会导致扇区0和扇区1被当作同一1024字节块的前后半读取，所有扇区号计算全部错位。

HAL开发者向队友指出了这个关键差异后，AI才正确重写了 sread()/swrite() 辅助函数，实现 blkno=sec/spb; offset=(sec%spb)*bps 的转换（其中spb=BSIZE/512=2）。

D.3.2 AI在FAT32中的其他局限

| 问题 | AI表现 | 实际解决 |
|-----------------|-----------------------------------|--|
| BPB_TotSec16兼容性 | AI按FAT32规范强制要求 TotSec16=0，拒绝挂载小镜像 | 队友实测发现Linux mkfs.fat对≤32MB镜像将总扇区数填入 TotSec16。改为 TotSec16?TotSec16:TotSec32 |
| mount失败回滚 | AI初版mount路径错误处理不完整 | 队友补充了 mp->ops->unmount(mp)+kfree(mp) 的完整回滚 |
| 镜像大小 | AI生成10MB镜像 | 足够测试使用，无需修改 |

D.3.3 LoongArch FAT32适配：经实测确认不可行

AI尝试了三种方案使LoongArch支持FAT32（超小镜像NDISK=1、双ramdisk NDISK=2、三ramdisk NDISK=3）。实测方案B（双ramdisk）后发现：kernel.bin = 4.26MB > 4MB QEMU ROM限制。结论：在现有ramdisk架构下，必须为LoongArch实现PCI virtio驱动才能支持FAT32，赛前时间不允许。